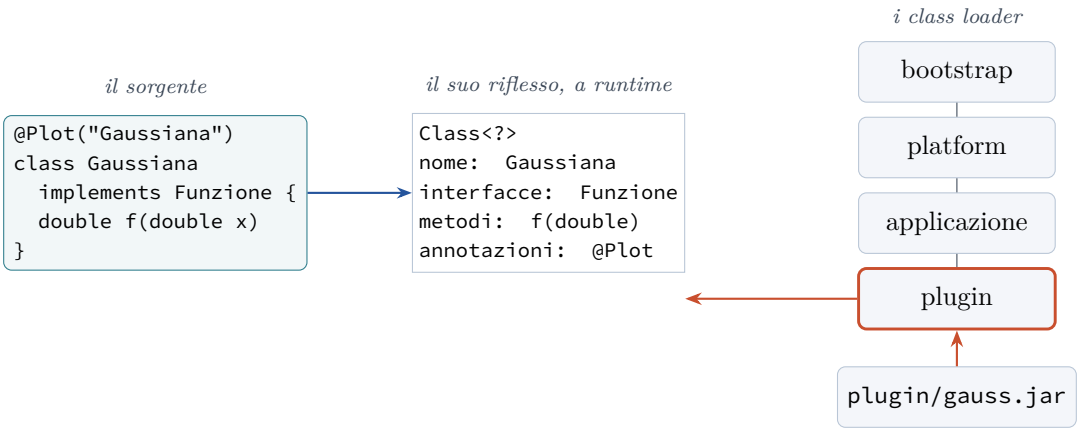
PAJC · Programmazione Avanzata in Java e C · 2026/27

Dispensa extra 1

# Reflection e codice a runtime

## La classe come oggetto, ispezione e invocazione, annotazioni e proxy, class loader e plugin, il compilatore dentro il programma




Un programma che si guarda, e che accoglie codice scritto dopo.

**prof. Massimiliano Redolfi**

Università degli Studi di Brescia · Ingegneria Informatica

massimiliano.redolfi@unibs.it

Versione 0.1

Quest'opera è distribuita con licenza Creative Commons "Attribuzione – Non commerciale – Condividi allo stesso modo 4.0 Internazionale".

# Indice


**1 Introduzione: un menù che non basta mai** **1**
- 1.1 Il problema . . . 1
- 1.2 La classe come oggetto . . . 3
- 1.3 Che cosa serve . . . 4
- 1.4 La strada che percorreremo . . . 4
- Esercizi . . . 5
- Risposte . . . 5

**2 La classe come oggetto** **7**
- 2.1 Un tipo è anche un valore . . . 7
- 2.2 Che cosa dice di sé . . . 8
- 2.3 Ragionare sui tipi a tempo di esecuzione . . . 11
- 2.4 Caricamento, collegamento, inizializzazione . . . 12
- 2.5 L'esempio guida: una classe dal suo nome . . . 13
- Esercizi . . . 16
- Risposte . . . 18

**3 Dentro la classe: campi, metodi, costruttori** **19**
- 3.1 I membri come oggetti . . . 19
- 3.2 Invocare . . . 21
- 3.3 Costruire . . . 23
- 3.4 Leggere e scrivere i campi . . . 24
- 3.5 Il costo . . . 25
- 3.6 L'esempio guida: il menù si riempie da solo . . . 26
- Esercizi . . . 29
- Risposte . . . 31

**4 Annotazioni e proxy: come fanno i framework** **33**
- 4.1 Metadati nel codice . . . 33
- 4.2 Dichiararne una . . . 34
- 4.3 Leggerle . . . 35
- 4.4 Un mini-JUnit in quaranta righe . . . 36
- 4.5 Un oggetto senza classe: `Proxy` . . . 38
- 4.6 L'esempio guida: funzioni che si presentano . . . 41
- Esercizi . . . 44
- Risposte . . . 46

**5 Da dove vengono le classi: class loader, jar e plugin** **47**
- 5.1 Chi legge i `.class` . . . 47
- 5.2 Un loader che guarda altrove . . . 49
- 5.3 Un nome, due loader, due tipi . . . 49
- 5.4 Il `.jar` . . . 51
- 5.5 Plugin alla maniera di Java: `ServiceLoader` . . . 52

5.6 L'esempio guida: la cartella `plugin/` . . . 52
Esercizi . . . 55
Risposte . . . 57

**6 Codice scritto adesso: il compilatore dentro il programma 58**
6.1 `javac` è una classe . . . 58
6.2 Leggere gli errori . . . 62
6.3 Il modello . . . 63
6.4 Tutto in memoria . . . 65
6.5 Dal codice al `.jar` . . . 67
6.6 L'esempio guida: la funzione scritta nel campo . . . 68
Esercizi . . . 70
Risposte . . . 72

**7 Palestra 74**
Problemi . . . 74

**8 Dove andare adesso 78**
8.1 `java.lang.invoke`: chiamate scoperte a runtime, veloci come le dirette . . . 78
8.2 Classi nascoste e bytecode generato . . . 78
8.3 Agenti e `Instrumentation` . . . 79
8.4 Il sistema dei moduli . . . 79
8.5 Dove la reflection lavora per noi, e dove non lavora più . . . 79
8.6 Java chiama il C . . . 79
Nelle altre dispense . . . 80
Riferimenti . . . 80

Capitolo 1

# Introduzione: un menù che non basta mai

## 1.1 Il problema

Il programma da cui parte questa dispensa disegna il grafico di una funzione. Una finestra, in alto una barra con un menù a tendina e due campi per l'intervallo delle $x$, al centro un pannello che campiona la funzione un punto per pixel e traccia la curva, con la griglia e gli assi. Il pannello, `PnlPlot`, è un componente Swing che si disegna da solo e che qui diamo per scritto: riceve una funzione da `double` a `double` con `setFunzione` e un intervallo con `setIntervallo`, e fa tutto il resto. La funzione è un `DoubleUnaryOperator`, cioè una lambda da un `double` a un `double`, e il menù è una mappa da un nome alla sua lambda:

```
public class PlotApp {

    private static final Map<String, DoubleUnaryOperator> FUNZIONI =
            new LinkedHashMap<>();
    static {
        FUNZIONI.put("sin(x)", Math::sin);
        FUNZIONI.put("cos(x)", Math::cos);
        FUNZIONI.put("x^2", x -> x * x);
        FUNZIONI.put("x^3 - 3x", x -> x * x * x - 3 * x);
        FUNZIONI.put("exp(-x^2/2)", x -> Math.exp(-x * x / 2));
        FUNZIONI.put("sin(x) / x", x -> Math.sin(x) / x);
    }

    private final JFrame frame = new JFrame("Plot di funzioni");
    private final PnlPlot pnlPlot = new PnlPlot();

    public PlotApp() {
        String[] nomi = FUNZIONI.keySet().toArray(new String[0]);
        JComboBox<String> cmbFunzione = new JComboBox<>(nomi);
        cmbFunzione.addActionListener(e ->
            pnlPlot.setFunzione(FUNZIONI.get(cmbFunzione.getSelectedItem())));
        JPanel barra = new JPanel(new FlowLayout(FlowLayout.LEFT));
        barra.add(new JLabel("f(x) ="));
        barra.add(cmbFunzione);
        frame.add(barra, BorderLayout.NORTH);
        frame.add(pnlPlot, BorderLayout.CENTER);
        pnlPlot.setFunzione(FUNZIONI.get("sin(x)"));
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(900, 650);
    }

    public static void main(String[] args) {
        EventQueue.invokeLater(() -> new PlotApp().frame.setVisible(true));
    }
}
```

Funziona, ed è anche scritto bene: aggiungere una funzione è una riga nella mappa. Ma è una riga *nel sorgente*. Chi usa il programma, e non lo scrive, ha davanti sei funzioni e nessun modo di averne una settima. La prima persona che lo prova chiede tre cose, in quest'ordine.

1. **Scriverla.** Un campo di testo accanto al menù in cui digitare `sin(x) * exp(-x / 5)` e premere *Disegna*. Non una notazione nostra da interpretare con un parser scritto apposta: la stessa sintassi di Java, con `Math` a disposizione, così che tutto ciò che si può scrivere in una lambda si possa scrivere anche lì.

2. **Salvarla e condividerla.** Una volta che la funzione è giusta, un pulsante *Salva* che la mette in un file; il file lo si manda a un collega, che lo copia in una cartella e se la ritrova nel menù, senza toccare il codice e senza ricompilare niente.

3. **Una cartella `plugin/`.** Lo stesso meccanismo, ma per funzioni scritte in Java da zero da chiunque: una classe con un metodo `f(double)`, magari con un nome e un intervallo di default, compilata e impacchettata in un `.jar`; il programma, all'avvio, guarda nella cartella e carica tutto quello che trova.

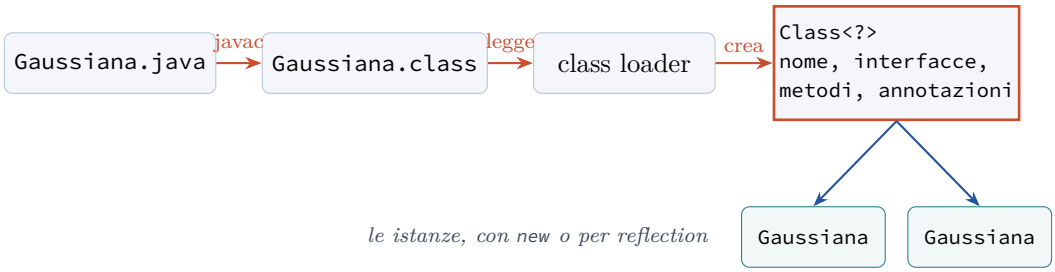
Il primo riflesso di chi ha già programmato è un `switch` sulla stringa, o una tabella di nomi e lambda più lunga. Non risolve nulla: i casi dello `switch` sono scritti nel sorgente esattamente come le righe della mappa, e una funzione che nessuno ha previsto non ci sarà mai. Il secondo riflesso è scrivere un piccolo interprete di espressioni; è un esercizio istruttivo, ma reinventa un pezzo di compilatore per una sintassi più povera di quella che già abbiamo, e non risponde alla terza richiesta, dove il codice è Java vero. La strada giusta passa da tre domande, che sono anche i tre temi della dispensa.

- Un programma in esecuzione, come fa a *sapere* che cosa c'è in una classe (che metodi ha, che interfacce implementa, che annotazioni porta) e a *usarla* se, quando è stato scritto, quella classe non esisteva e il suo nome è arrivato come stringa?
- Come si *carica* codice compilato che non c'era quando il programma è stato compilato: un `.class` in una cartella, un `.jar` messo lì da qualcun altro?
- Come si *compila* codice scritto adesso, dall'utente, in un campo di testo, e lo si esegue dentro il programma come se fosse sempre stato suo?

Alla fine della dispensa, nel capitolo 7, il Plot farà tutte e tre le cose; i capitoli in mezzo costruiscono gli strumenti, uno per volta, e li provano sul Plot stesso.

## 1.2 La classe come oggetto

Tutto quello che segue gira attorno a un'idea sola, che in Java è vera dal primo giorno ma che di solito non si guarda mai in faccia: **una classe, a tempo di esecuzione, è un oggetto**. Non un'astrazione del compilatore che sparisce dopo `javac`, ma un oggetto nell'heap, di tipo `java.lang.Class`, che descrive la classe (il nome, la superclasse, le interfacce, i campi, i metodi, i costruttori, le annotazioni) e da cui, volendo, si creano le istanze. Il cammino che porta dal sorgente a quell'oggetto è questo:




Dal sorgente all'oggetto `Class`: compilazione, caricamento, e da lì le istanze.

Il compilatore trasforma il sorgente in un file `.class`, bytecode più una tabella che descrive tutto ciò che la classe dichiara. Quando il programma nomina la classe per la prima volta, un *class loader* legge quel file e costruisce l'oggetto `Class` che la rappresenta: uno solo, per quella classe, per quel loader. Da quell'oggetto discendono le istanze, sia con `new`, che il compilatore risolve in anticipo, sia a tempo di esecuzione, chiedendolo all'oggetto stesso.

Le tre domande della sezione precedente sono tre modi di girare attorno a questo oggetto. La prima chiede che cosa dice di sé e che cosa gli si può far fare: è la *reflection*, il programma che si guarda allo specchio (capitoli 2, 3 e 4). La seconda chiede chi lo costruisce e da dove prende i byte: sono i class loader, e con loro i `.jar` e i plugin (capitolo 5). La terza chiede come se ne fabbrica uno nuovo mentre il programma gira: è il compilatore chiamato come una libreria, con il loader subito dopo (capitolo 6).

**Osservazione 1.1**

Finora il tipo di ogni cosa era deciso quando si scrive il programma, e il compilatore lo verificava: un metodo che non esiste è un errore prima ancora di eseguire. La reflection sposta quelle decisioni a tempo di esecuzione: il nome del metodo può arrivare da una stringa, la classe da un file che non c'era. È ciò che rende possibile il Plot che vogliamo, ed è anche ciò che rende la reflection pericolosa: quello che il compilatore non vede, non lo controlla nessuno, e un errore che prima era una riga rossa nell'editor diventa un'eccezione mentre il programma gira. Ogni capitolo tornerà su questo prezzo, e sulle regole per pagarlo il meno possibile.

## 1.3 Che cosa serve

La dispensa dà per acquisite le basi del linguaggio: classi, interfacce, eccezioni, collezioni, lettura e scrittura di file, tutte in *Java Fast Forward*. Le lambda e le interfacce funzionali, in particolare `DoubleUnaryOperator`, e le method reference come `Math::sin` sono in *Dal nome alla funzione*; le useremo per incartare in una funzione ciò che la reflection ci consegna come oggetto `Method`. Le interfacce con metodi `default`, le gerarchie `sealed` e i record sono in *Le interfacce in Java moderno*: nel capitolo 2 li guarderemo dal lato di `Class`, e nel capitolo 5 un'interfaccia sarà il contratto che lega il programma ai suoi plugin. Gli stream (*Stream, per intero*) compaiono per filtrare e ordinare gli array di metodi che la reflection restituisce.

Il Plot è un'interfaccia grafica, ma la dispensa non insegna Swing: una finestra con un campo, un pulsante e un pannello che si disegna sono in *Interfacce grafiche con Swing*, e il pannello del plot è allegato alle soluzioni, pronto da usare. Nel capitolo 6 compilare una classe costa qualche decimo di secondo e va fatto fuori dal thread dell'interfaccia: il perché e il come sono in *Thread e concorrenza*, e qui basta un rimando.

Restano fuori, di proposito, le cose che la reflection rende possibili ma che meritano un libro ciascuna: `java.lang.invoke` e i *method handle*, oltre un assaggio nel capitolo 8; la generazione di bytecode; gli agenti; il sistema dei moduli, oltre quanto serve a capire perché certe cose non si possono più fare; la serializzazione. E resta fuori, perché in Java 21 non esiste più, una *sandbox* in cui far girare il codice dell'utente senza rischi: nel capitolo 6 diremo che cosa comporta e che cosa si fa invece.

## 1.4 La strada che percorreremo

Il capitolo 2 presenta l'oggetto `Class`: le tre porte per ottenerlo (`.class`, `getClass()`, `Class.forName`), che cosa dice di sé, come si ragiona sui tipi a tempo di esecuzione, e che cosa succede davvero quando una classe viene caricata e inizializzata. Il Plot guadagna un campo in cui scrivere il nome qualificato di una classe che implementa `Funzione`, e la disegna.

Il capitolo 3 entra nella classe: campi, metodi e costruttori come oggetti, la differenza fra ciò che è dichiarato e ciò che è pubblico, l'invocazione con `invoke` e le sue trappole, l'accesso ai membri privati e i suoi limiti, il costo di tutto questo misurato. Il menù del Plot si riempie da solo con le funzioni di `Math`, scoperte per reflection.

Il capitolo 4 è dedicato ai metadati e a chi li legge: dichiarare un'annotazione, leggerla, costruirci sopra un piccolo framework di test; poi i proxy dinamici, oggetti senza classe che intercettano ogni chiamata. Le funzioni del Plot portano un'annotazione con il nome e l'intervallo, e un proxy conta le valutazioni.

Il capitolo 5 risponde alla seconda domanda: i class loader, la catena di delega, un loader che guarda in una cartella o in un `.jar`, la regola per cui la stessa classe caricata due volte è

due tipi diversi, il meccanismo standard dei servizi. Il Plot ha la sua cartella `plugin/` e un pulsante *Ricarica*.

Il capitolo 6 risponde alla terza: il compilatore di Java è una classe della libreria standard, e si può chiamare con un sorgente in una stringa; gli errori tornano come oggetti, i `.class` vanno dove diciamo noi, e un loader li carica. Il Plot compila l'espressione scritta nel campo e salva la funzione come plugin.

Il capitolo 7 è la palestra: sette programmi senza teoria nuova, dai quesiti nello stile delle prove scritte fino al Plot completo. Il capitolo 8 indica dove continuare.

## Esercizi

**Esercizio 1.1: Dove sta la conoscenza (⋆)**

Nel `PlotApp` di apertura, dire in quale momento (scrittura del sorgente, compilazione, esecuzione) viene deciso: (a) quante funzioni ci sono nel menù; (b) che cosa calcola `x -> x * x`; (c) quale funzione è disegnata quando l'utente sceglie una voce; (d) che `Math::sin` è compatibile con `DoubleUnaryOperator`.

**Esercizio 1.2: Lo switch non basta (⋆)**

Un collega propone di soddisfare la prima richiesta dell'utente con un campo di testo e uno `switch` sul suo contenuto: `case "sin(x)" -> Math::sin`, e così via per una ventina di espressioni. Spiegare in due frasi perché non è una risposta alla richiesta, e che cosa succede quando l'utente scrive `sin(2*x)`.

**Esercizio 1.3: Tipo statico, tipo dinamico (⋆)**

Nel frammento seguente, dire qual è il tipo *statico* di `f` (quello che vede il compilatore) e qual è il tipo *dinamico* dell'oggetto a cui `f` punta a tempo di esecuzione, e perché la seconda informazione esiste solo quando il programma gira.

```
DoubleUnaryOperator f = Math::sin;
Object o = f;
```

## Risposte

**Risposta 1.1.** (a) Alla scrittura del sorgente: le `put` sono righe del programma, e la compilazione le fissa. (b) Alla scrittura, e la compilazione lo verifica: il corpo della lambda è codice. (c) All'esecuzione: la scelta dell'utente arriva come stringa e la mappa la risolve a runtime, ma solo fra le chiavi che esistevano alla compilazione. (d) Alla compilazione: è il compilatore a controllare che `Math.sin` abbia la firma di `applyAsDouble`. La dispensa sposta (a) e (b) a tempo di esecuzione, e scoprirà che anche (d) si può chiedere a runtime.

**Risposta 1.2.** Lo `switch` elenca le stesse funzioni della mappa, solo con una sintassi diversa: la conoscenza di che cosa si può disegnare resta nel sorgente, e l'utente può solo scegliere fra ciò che l'autore ha previsto. `sin(2*x)` non è in nessun `case` e finisce nel `default`: nessuna curva, o un messaggio d'errore, esattamente come prima.

**Risposta 1.3.** Il tipo statico di `f` è `DoubleUnaryOperator`; quello di `o` è `Object`. L'oggetto a cui entrambi puntano è un'istanza di una classe generata dalla JVM per quella method reference, con un nome del tipo `PlotApp$$Lambda/0x...`, che nel sorgente non compare da nessuna parte: esiste solo quando il programma gira, ed è quello che `o.getClass()` restituisce. Il capitolo 2 parte da qui.

Capitolo 2

# La classe come oggetto

Il capitolo 1 ha messo al centro un'idea: a tempo di esecuzione ogni classe è rappresentata da un oggetto, di tipo `java.lang.Class`, e tutto ciò che la reflection sa fare parte da lì. Questo capitolo è dedicato a quell'oggetto: come lo si ottiene, che cosa dice della classe che rappresenta, come ci si ragiona sui tipi quando il compilatore non può più aiutarci, e che cosa succede davvero nel momento in cui la JVM lo costruisce. Alla fine il Plot avrà un campo in cui scrivere il nome di una classe, e la disegnerà: è il primo passo verso la prima delle tre richieste del capitolo 1, la funzione scelta dall'utente e non dall'autore.

## 2.1 Un tipo è anche un valore

In tutto ciò che abbiamo scritto finora i tipi stavano da una parte sola della riga: a sinistra, nelle dichiarazioni, o dentro `new`, `instanceof` e i cast. Un tipo non era mai un *valore* da mettere in una variabile, passare a un metodo, confrontare. In Java lo è, e la classe che lo rappresenta si chiama `Class`. Un'espressione di tipo `Class<Circle>` vale l'oggetto che descrive la classe `Circle`; l'argomento di tipo dice quale classe, quando lo si sa, e `Class<?>` ("una classe qualunque") quando non lo si sa. Ci sono tre porte per arrivare all'oggetto, e corrispondono a tre momenti diversi in cui si decide *quale* classe:

```
Class<Circle> a = Circle.class;                  // il tipo scritto nel sorgente
Shape2d s = new Circle(1);
Class<? extends Shape2d> b = s.getClass();       // il tipo dell'oggetto, a
    runtime
Class<?> c = Class.forName("bimondo.Circle");    // da una stringa
System.out.println(a == b);
System.out.println(b == c);
```

```
true
true
```

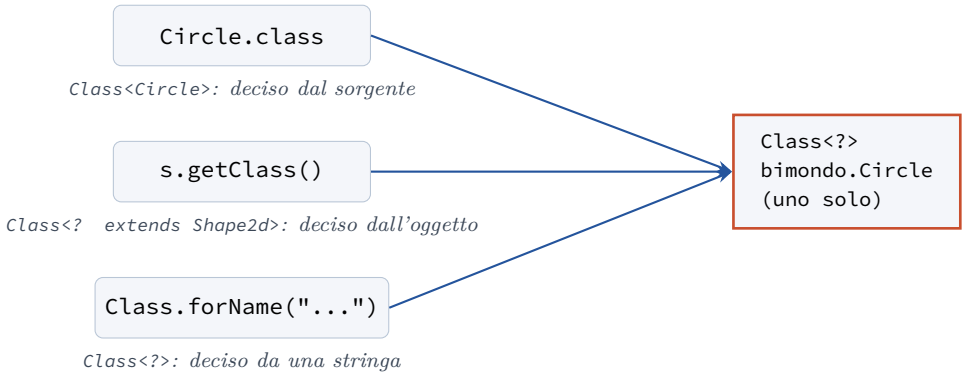
La prima porta è il *letterale di classe* `Circle.class`: il nome della classe seguito da `.class`. La classe deve esistere quando si compila, e il risultato ha il tipo più preciso, `Class<Circle>`. La seconda è `getClass()`, un metodo di `Object` che ogni oggetto possiede: restituisce la classe *dell'oggetto*, cioè il suo tipo dinamico, non il tipo della variabile; per questo su una variabile `Shape2d` che contiene un `Circle` si ottiene `Circle`, e il tipo del risultato è `Class<? extends Shape2d>`, "una sottoclasse di `Shape2d`, non so quale". La terza è `Class.forName`, che riceve il nome come stringa: il compilatore non può sapere se quella classe esiste, e per questo il metodo dichiara l'eccezione controllata `ClassNotFoundException` e restituisce un `Class<?>`.

I due `true` dicono una cosa importante: le tre porte danno lo *stesso* oggetto. Per ogni classe esiste un solo oggetto `Class` (per ogni class loader, precisazione che diventerà centrale nel capitolo 5), e quindi due `Class` si confrontano con `==`, senza `equals`.

**Definizione 2.1: Nome binario**

Il nome che `Class.forName` e `getName()` usano per una classe è il suo *nome binario*: il nome qualificato con il package, con il segno `$` fra una classe esterna e una annidata (`java.util.Map$Entry`), e una notazione propria per gli array: `[I` per `int[]`, `[[Ljava.lang.String;` per `String[][]`. `getSimpleName()` dà il nome senza package (`Entry`, `int[]`), `getTypeName()` il nome come lo si scrive nel sorgente (`java.util.Map$Entry`, `int[]`).

I tipi primitivi hanno anch'essi il loro oggetto `Class`, che si ottiene con il letterale (`int.class`) o con la costante `Integer.TYPE`, e che è *diverso* da `Integer.class`: `int.class == Integer.class` è `false`, e la differenza conta quando si cerca un metodo per firma (capitolo 3). Hanno un `Class` anche gli array, uno per tipo di elemento e numero di dimensioni, e le lambda: una lambda è un'istanza di una classe che il compilatore non scrive e la JVM genera, con un nome del tipo `Porte$$Lambda/0x0000400001040c00`, diverso a ogni esecuzione, che `getClass().getName()` mostra senza imbarazzo.




Tre porte, un oggetto: il momento in cui si sceglie la classe è diverso, il risultato no.

**Osservazione 2.1**

La porta che rende possibile tutta la dispensa è la terza. `Circle.class` e `getClass()` non dicono nulla che il programma non sapesse già; `forName` accetta un nome che può arrivare da un campo di testo, da un file di configurazione, da una riga di comando, e per una classe che non esisteva quando il programma è stato scritto. Il prezzo è nell'eccezione: `ClassNotFoundException` è il primo errore che prima era del compilatore e adesso è nostro.

## 2.2 Che cosa dice di sé

Una volta in mano, l'oggetto `Class` risponde a tutte le domande che si farebbero leggendo la prima riga della dichiarazione: come si chiama, in che package sta, se è una classe, un'interfaccia, un record, un enum, un array; che modificatori ha; che cosa estende e che cosa implementa; se è `sealed`, chi può estenderla. Il programma che segue, `Scheda`, stampa tutto questo per una classe qualunque, e lo proviamo sul bimondo: un'interfaccia `Shape2d` dichiarata `sealed` con due record, `Circle` e `Rectangle`, che la implementano.

```
public class Scheda {

    public static void stampa(Class<?> c) {
        System.out.println("nome:         " + c.getName());
        System.out.println("semplice:     " + c.getSimpleName());
        System.out.println("package:      " + c.getPackageName());
        System.out.println("modificatori: " +
    Modifier.toString(c.getModifiers()));
        System.out.println("tipo:         " + tipo(c));
        System.out.println("superclasse:  " + c.getSuperclass());
        System.out.println("interfacce:   " + Arrays.toString(c.getInterfaces()));
        if (c.isRecord()) {
            System.out.print("componenti:  ");
            for (RecordComponent rc : c.getRecordComponents())
                String tipo = rc.getType().getSimpleName();
                System.out.print(" " + tipo + " " + rc.getName());
            System.out.println();
        }
        if (c.isSealed())
            List<Class<?>> permesse = List.of(c.getPermittedSubclasses());
            System.out.println("permesse:     " + permesse);
    }

    private static String tipo(Class<?> c) {
        if (c.isInterface()) return c.isSealed() ? "interfaccia sealed" :
    "interf.";
        if (c.isRecord()) return "record";
        if (c.isEnum()) return "enum";
        if (c.isArray()) return "array di " +
    c.getComponentType().getSimpleName();
        if (c.isPrimitive()) return "primitivo";
        return "classe";
    }
}
```

`getModifiers()` restituisce un intero in cui ogni bit è un modificatore (`public`, `final`, `abstract`, `static`…); la classe `Modifier` lo traduce in parole con `toString` e lo interroga bit per bit con `isPublic`, `isFinal` e simili. Le interfacce portano il bit `abstract` anche se nessuno lo scrive, e gli array risultano `abstract final`: sono dettagli della rappresentazione interna, e la scheda li mostra com'è. Ecco l'output per l'interfaccia, per `Circle` e per `int[]`:

```
nome:          bimondo.Shape2d
semplice:      Shape2d
package:       bimondo
modificatori: public abstract interface
tipo:          interfaccia sealed
superclasse:   null
interfacce:    []
permesse:      [class bimondo.Circle, class bimondo.Rectangle]

nome:          bimondo.Circle
semplice:      Circle
package:       bimondo
modificatori: public final
tipo:          record
superclasse:   class java.lang.Record
interfacce:    [interface bimondo.Shape2d]
componenti:    double radius

nome:          [I
semplice:      int[]
package:       java.lang
modificatori: public abstract final
tipo:          array di int
superclasse:   class java.lang.Object
interfacce:    [interface java.lang.Cloneable, interface java.io.Serializable]
```

Si notano alcune cose che dal sorgente non si vedono. Un'interfaccia non ha superclasse (`null`), e le interfacce che estende si ottengono comunque da `getInterfaces()`. Un record è una classe `final` che estende `java.lang.Record`, e i suoi componenti, con nome e tipo, sono disponibili attraverso `getRecordComponents()` JAVA 16; è così che una libreria può convertire un record in una riga di testo o in un documento JSON senza sapere niente di quel record in particolare. Un'interfaccia `sealed` JAVA 17 espone con `getPermittedSubclasses()` esattamente la lista che il sorgente scrive dopo `permits`: il programma può scoprire a runtime l'intera gerarchia chiusa, e un `switch` esaustivo su quella gerarchia (*Le interfacce in Java moderno*) ha qui la sua controparte riflessiva.

**Attenzione 2.1: `getSuperclass()` può essere `null`**

Per `Object`, per le interfacce e per i tipi primitivi `getSuperclass()` restituisce `null`, e una scheda scritta con `c.getSuperclass().getSimpleName()` lancia `NullPointerException` proprio sulle interfacce, che sono i tipi su cui più spesso si interroga la reflection. Chi risale una gerarchia con un ciclo `while (c != null) c = c.getSuperclass()` lo fa bene; chi scrive la catena di chiamate senza controllo lo scopre a runtime, perché il compilatore non vede nulla di strano in una chiamata su un riferimento che può essere nullo.

## 2.3 Ragionare sui tipi a tempo di esecuzione

Con un `Class<?>` in mano arrivano le domande sulle relazioni fra tipi, che finora facevamo solo con `instanceof` e con i cast, cioè con il tipo scritto nel sorgente. Le loro versioni riflessive prendono il tipo come oggetto:

```
Object o = new Circle(2);
System.out.println(o instanceof Shape2d);                          // true
System.out.println(Shape2d.class.isInstance(o));                     // true
System.out.println(Shape2d.class.isAssignableFrom(Circle.class));  // true
System.out.println(Circle.class.isAssignableFrom(Shape2d.class));  // false
Shape2d s = Shape2d.class.cast(o);                                 // come
    (Shape2d) o
Class<? extends Shape2d> k = Circle.class.asSubclass(Shape2d.class);
```

`isInstance(o)` è `instanceof` con il tipo a destra deciso a runtime; `isAssignableFrom(k)` è la stessa domanda fra due classi, "un valore di tipo `k` si può assegnare a una variabile di questo tipo?", e si legge al contrario di come si scrive: `A.isAssignableFrom(B)` è vero se `B` è `A` o una sua sottoclasse. `cast(o)` fa il cast e lancia `ClassCastException` se non può, esattamente come il cast scritto, ma con il tipo scelto a runtime; `asSubclass(T.class)` fa la stessa cosa su un `Class`, e restituisce un `Class<? extends T>` che d'ora in poi il compilatore tratta come tale. È lo strumento con cui, nel Plot, dopo aver verificato che la classe dell'utente implementa `Funzione`, si ottiene un riferimento tipato invece di un `Class<?>` qualunque.

**Attenzione 2.2: Il cast "per far tacere il compilatore"**

Scrivere `(Funzione) classe.getDeclaredConstructor().newInstance()` senza prima aver controllato `Funzione.class.isAssignableFrom(classe)` compila lo stesso, e funziona finché l'utente scrive il nome di una classe giusta. Con il nome di una classe sbagliata, per esempio `java.lang.String`, il cast fallisce con una `ClassCastException` a runtime, nel mezzo di un listener, con un messaggio che parla di `String` e di `Funzione` e che l'utente non capisce. Il controllo prima del cast non cambia ciò che il programma può fare: cambia chi riceve l'errore e che cosa gli si può dire.

Un limite va detto subito, perché torna ogni volta che la reflection incontra i generici. Il tipo `List<String>` esiste solo per il compilatore: a runtime la classe è `java.util.List` e basta, perché i parametri di tipo vengono cancellati durante la compilazione (*erasure*). Ne segue che `List<String>.class` non si può scrivere, che `new List<String>().getClass()` e `new List<Integer>().getClass()` darebbero lo stesso oggetto, e che per reflection non c'è modo di sapere di che tipo sono gli elementi di una lista guardando la lista. Resta una traccia nelle *dichiarazioni*: un campo dichiarato `List<String> nomi` conserva il suo tipo generico nel `.class`, e `Field.getGenericType()` lo restituisce; ma è il tipo scritto nel sorgente, non quello degli oggetti. Chi deve serializzare una lista a runtime lo chiede a chi gliela dà, non alla lista.

## 2.4 Caricamento, collegamento, inizializzazione

L'oggetto `Class` non c'è dall'inizio: la JVM lo costruisce quando il programma nomina la classe per la prima volta, in tre fasi che hanno conseguenze visibili. Il *caricamento* legge il `.class` e crea l'oggetto; il *collegamento* verifica il bytecode e prepara la memoria per i campi statici, con i valori di default; l'*inizializzazione* esegue, nell'ordine in cui compaiono nel sorgente, gli inizializzatori dei campi statici e i blocchi `static { }`. Le prime due fasi sono invisibili; la terza esegue codice nostro, e il momento in cui avviene è fissato dalla specifica: la prima volta che si crea un'istanza, si chiama un metodo statico, si legge o si scrive un campo statico non costante, o si chiama `Class.forName`. Il programma seguente lo mostra con una classe che annuncia la propria inizializzazione:

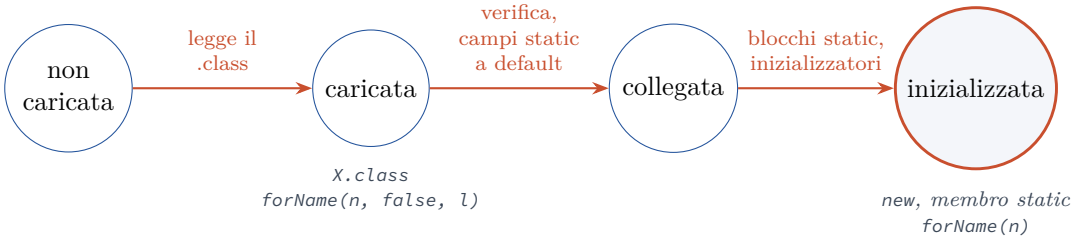
```
class Configurazione {
    static final String VERSIONE = "1.0";          // costante: non inizializza
    static final List<String> VOCI = new ArrayList<>();
    static {
        System.out.println("  [inizializzo Configurazione]");
        VOCI.add("default");
    }
    public static void saluta() { System.out.println("  ciao da Configurazione");
    }
}

public class Inizializzazione {
    public static void main(String[] args) throws Exception {
        System.out.println("1. riferimento al tipo:");
        Class<?> c = Configurazione.class;
        System.out.println("2. forName con initialize=false:");
        ClassLoader loader = Inizializzazione.class.getClassLoader();
        Class.forName("bimondo.Configurazione", false, loader);
        System.out.println("3. una costante:");
        System.out.println("  " + Configurazione.VERSIONE);
        System.out.println("4. un metodo statico:");
        Configurazione.saluta();
        System.out.println("5. forName di nuovo:");
        Class.forName("bimondo.Configurazione");
        System.out.println("fine " + c.getSimpleName());
    }
}
```

```
1. riferimento al tipo:
2. forName con initialize=false:
3. una costante:
  1.0
4. un metodo statico:
  [inizializzo Configurazione]
  ciao da Configurazione
5. forName di nuovo:
fine Configurazione
```

Il blocco statico gira una volta sola, al passo 4, e nessuno dei passi precedenti lo ha

provocato. Il letterale `Configurazione.class` dà l'oggetto senza inizializzare la classe; la variante di `forName` a tre argomenti, con `initialize` a `false` e il loader da usare, carica e collega ma non inizializza, e serve proprio quando si vuole ispezionare una classe senza far girare il suo codice (un plugin di cui non ci si fida ancora, nel capitolo 5). La costante `VERSIONE` è un caso a parte: un campo `static final` di tipo primitivo o `String` inizializzato con un letterale viene *copiato* dal compilatore in chi lo usa, e leggerlo non tocca la classe. Infine `forName` a un argomento, al passo 5, inizializza, ma la classe lo è già: non succede nulla.




Le tre fasi di una classe; solo l'ultima esegue codice nostro, e una volta sola.

Le eccezioni raccontano in quale fase qualcosa è andato storto. `ClassNotFoundException` è un'eccezione controllata che arriva da `forName` quando il loader non trova nessun file con quel nome: il nome è sbagliato, o il `.class` non è nel class path. `NoClassDefFoundError` è invece un errore, non controllato, che arriva quando il codice già compilato nomina una classe che al momento della compilazione c'era e adesso non c'è più (una libreria tolta dal class path): è il compilatore che l'aveva vista, e la JVM che non la trova. `ExceptionInInitializerError` incarta un'eccezione lanciata da un blocco statico: la classe resta in uno stato irrecuperabile, e ogni uso successivo fallisce con `NoClassDefFoundError`, un sintomo che senza questa pagina appare inspiegabile.

**Osservazione 2.2**

Un blocco statico che può fallire è un rischio nascosto: il primo uso della classe, in qualunque punto del programma, diventa il punto in cui esplode. Per questo le librerie evitano di fare lavoro vero nei blocchi statici (aprire file, leggere configurazioni) e preferiscono un'inizializzazione esplicita, e per questo il capitolo 5 ispezionerà i plugin con `initialize` a `false` prima di decidere se usarli.

## 2.5 L'esempio guida: una classe dal suo nome

Torniamo al Plot. Per ora non abbiamo un compilatore né una cartella di plugin, ma abbiamo abbastanza per il primo passo: invece del menù fisso, un campo in cui l'utente scrive il nome qualificato di una classe, e il programma la carica, verifica che sia una funzione, la istanzia e la disegna. Serve un contratto, cioè un'interfaccia che dica che cosa è una funzione per il Plot:

```
public interface Funzione {
    double f(double x);
}
```

Le funzioni sono classi che la implementano, per esempio in un package `funzioni`:

```
public class Gaussiana implements Funzione {
    public double f(double x) {
        return Math.exp(-x * x / 2);
    }
}
```

E la finestra, al posto del menù, ha il campo, un pulsante *Carica* e una riga di stato in basso:

```
public class PlotApp {

    private final JFrame frame = new JFrame("Plot di funzioni");
    private final PnlPlot pnlPlot = new PnlPlot();
    private final JTextField txtClasse =
            new JTextField("it.unibs.pajc.plot.funzioni.Gaussiana", 34);
    private final JLabel lblStato = new JLabel(" ");

    public PlotApp() {
        JButton btnCarica = new JButton("Carica");
        btnCarica.addActionListener(e -> carica());
        txtClasse.addActionListener(e -> carica());
        JPanel barra = new JPanel(new FlowLayout(FlowLayout.LEFT));
        barra.add(new JLabel("classe:"));
        barra.add(txtClasse);
        barra.add(btnCarica);
        frame.add(barra, BorderLayout.NORTH);
        frame.add(pnlPlot, BorderLayout.CENTER);
        frame.add(lblStato, BorderLayout.SOUTH);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(900, 650);
        carica();
    }
```

La finestra è quella del capitolo 1 con il campo al posto del menù; `carica()` è chiamato dal pulsante, dal tasto Invio nel campo e una volta all'avvio, ed è tutto il capitolo in venti righe:

```
    private void carica() {
        String nome = txtClasse.getText().trim();
        try {
            Class<?> classe = Class.forName(nome);
            if (!Funzione.class.isAssignableFrom(classe)) {
                lblStato.setText(nome + " non implementa Funzione");
                return;
            }
            Funzione fn = (Funzione)
    classe.getDeclaredConstructor().newInstance();
            pnlPlot.setFunzione(fn::f);
            lblStato.setText(scheda(classe));
        } catch (ClassNotFoundException e) {
            lblStato.setText("classe non trovata: " + nome);
        } catch (ReflectiveOperationException e) {
            lblStato.setText("impossibile creare un'istanza: " + e);
        }
    }

    private static String scheda(Class<?> c) {
        return c.getSimpleName() + " (" + c.getPackageName() + "), "
                + (c.isRecord() ? "record" : "classe") + ", estende "
                + c.getSuperclass().getSimpleName() + ", implementa "
                + Arrays.toString(c.getInterfaces());
    }

    public static void main(String[] args) {
        EventQueue.invokeLater(() -> new PlotApp().frame.setVisible(true));
    }
}
```

Il nome arriva come stringa dal campo; `forName` lo risolve, o lancia `ClassNotFoundException` che diventa un messaggio nella riga di stato. `isAssignableFrom` fa la domanda giusta prima del cast: "un'istanza di questa classe è una `Funzione`?". L'istanza nasce dalla riga `classe.getDeclaredConstructor().newInstance()`, che chiede alla classe il suo costruttore senza argomenti e lo chiama: è materia del capitolo 3, e per ora basta sapere che può fallire in modi diversi (costruttore assente, non accessibile, che lancia un'eccezione), tutti sottoclassi di `ReflectiveOperationException`, che è anche la superclasse di `ClassNotFoundException`: per questo i due `catch` stanno in quell'ordine. Il cast a `Funzione` è sicuro perché l'abbiamo verificato; da lì in poi `fn::f` è un `DoubleUnaryOperator` come quelli del menù di partenza, e il pannello non sa che cosa è cambiato. La scheda nella riga di stato dice che cosa è stato caricato, e usa `getSuperclass()` senza controllo perché a quel punto sappiamo che `classe` non è un'interfaccia: implementa `Funzione`, ed è istanziabile.

**Ricetta: da un nome a un oggetto**

1. `Class.forName(nome)`: la classe esiste? (`ClassNotFoundException`);
2. `Contratto.class.isAssignableFrom(classe)`: è del tipo che ci serve? (altrimenti un messaggio, non un cast);
3. `classe.getDeclaredConstructor().newInstance()`: un'istanza (`ReflectiveOperationException`);
4. il cast al contratto, ormai sicuro, e da lì codice normale.

Con questa versione l'utente può scrivere `it.unibs.pajc.plot.funzioni.Sinc` e vedere la curva, ma solo per classi che stanno già nel programma: `forName` cerca nel class path del Plot, e una classe scritta da un collega non c'è. Il capitolo 3 resta ancora dentro il programma e sfrutta la reflection per riempire il menù da solo; il capitolo 5 insegnerà a `forName` a guardare altrove.

## Esercizi

**Esercizio 2.1: Tre porte (⋆)**

Dire, per ciascuna espressione, il tipo statico del risultato e se può lanciare un'eccezione: (a) `String.class`; (b) `"ciao".getClass()`; (c) `Class.forName("java.lang.String")`; (d) `((Object) "ciao").getClass()`. Le quattro espressioni danno lo stesso oggetto?

**Esercizio 2.2: Nomi (⋆)**

Scrivere che cosa stampano `getName()`, `getSimpleName()` e `getTypeName()` per `Map.Entry.class`, `double[].class` e `String[][].class`. Quale dei tre nomi accetta `Class.forName`?

**Esercizio 2.3: Assegnabile (⋆)**

Con `Shape2d`, `Circle` e `Rectangle` del capitolo, dire il valore di: (a) `Shape2d.class.isAssignableFrom(Circle.class);` (b) `Circle.class.isAssignableFrom(Circle.class);` (c) `Object.class.isAssignableFrom(Shape2d.class);` (d) `Circle.class.isInstance(new Rectangle(1, 2));` (e) `Number.class.isAssignableFrom(int.class).`

**Esercizio 2.4: Quando gira (⋆⋆)**

Con la classe `Configurazione` del capitolo, dire se il blocco statico gira, e quando, in ciascun frammento eseguito da solo in un `main`: (a) `Configurazione c = null;` (b) `System.out.println(Configurazione.VERSIONE);` (c) `System.out.println(Configurazione.VOCI.size());` (d) `Class.forName("bimondo.Configurazione", false, null);` (e) `Class<?>[] k = { Configurazione.class };`

### Esercizio 2.5: Due errori (⋆⋆)

Un programma compila e gira; poi si cancella dal class path il `.jar` di una libreria che usa. Dire quale eccezione o errore si vede (a) se la libreria è nominata direttamente nel codice (`new LibreriaX()`); (b) se è caricata con `Class.forName("x.LibreriaX")`; (c) se la libreria c'è ma il suo blocco statico lancia `IllegalStateException` al primo uso, e il programma la usa una seconda volta.

*I problemi che seguono chiedono un programma completo, in ordine crescente di difficoltà. Le soluzioni sono nel repository delle soluzioni del corso, nel progetto e nel package indicati.*

### Esercizio 2.6: La scheda da riga di comando (⋆)

Scrivere un programma che riceve uno o più nomi binari di classi dalla riga di comando (per esempio `java.util.ArrayList java.time.Month java.util.Map$Entry int[]`) e stampa per ciascuno la scheda del capitolo, completata con: la catena delle superclassi fino a `Object`, tutte le interfacce implementate direttamente e indirettamente (senza duplicati), le costanti di un enum. Un nome che non esiste produce una riga di errore e non ferma gli altri.

Soluzione: progetto `sol-dxtr-01-c02`, package `it.unibs.pajc.redolfi.e06`.

### Attenzione 2.3: Le interfacce di un'interfaccia

`getInterfaces()` restituisce solo le interfacce dichiarate *direttamente* da quella classe; quelle ereditate dalla superclasse e quelle estese dalle interfacce stesse vanno cercate risalendo. Chi scrive il ciclo sulle sole superclassi si perde `Collection` e `Iterable` per `ArrayList`, e chi lo scrive senza un insieme stampa `Collection` tre volte.

### Esercizio 2.7: L'ordine di inizializzazione (⋆)

Scrivere tre classi `A`, `B` e `C`, ciascuna con un blocco statico che stampa il proprio nome, in cui `A` ha un campo statico di tipo `B` inizializzato con `new B()`, e `B` ha una costante `static final int` presa da `C`. Prevedere su carta l'ordine delle stampe per un `main` che esegue `new A()`, poi per uno che esegue solo `Class.forName("A", false, ...)` seguito da `A.class.getSuperclass()`, poi verificare.

Soluzione: progetto `sol-dxtr-01-c02`, package `it.unibs.pajc.redolfi.e07`.

### Esercizio 2.8: Compatibile (⋆⋆)

Scrivere `static boolean compatibile(Object o, String nomeTipo)`, che dice se `o` si può assegnare a una variabile del tipo nominato, accettando anche nomi di primitivi (`"int"`: vero per un `Integer`) e di array (`"java.lang.Object[]"`). Provarlo con una tabella di coppie e stampare i risultati; i nomi inesistenti valgono `false` senza eccezioni. *Suggerimento: i primitivi non hanno un nome binario che `forName` accetti, e `"int[]"` nemmeno; serve una piccola tabella, e per gli array la forma `[L...;`.*

Soluzione: progetto `sol-dxtr-01-c02`, package `it.unibs.pajc.redolfi.e08`.

**Esercizio 2.9: La fabbrica di forme (⋆⋆)**

Un file di testo descrive una scena, una forma per riga: il nome semplice della classe e le sue misure, come `Circle 2.5` o `Rectangle 3 4`. Scrivere un programma che legge il file, costruisce per ogni riga l'oggetto giusto con `Class.forName` sul nome completato con il package delle forme, verifica che sia uno `Shape2d`, lo crea con il costruttore che riceve i `double` della riga, e alla fine stampa l'area totale e la forma più grande. Una riga sbagliata (classe inesistente, numero di misure diverso da quello del costruttore) si segnala e si salta. *Suggerimento: il costruttore giusto si trova con `getConstructor` passando tanti `double.class` quante sono le misure.*

Soluzione: progetto `sol-dxtr-01-c02`, package `it.unibs.pajc.redolfi.e09`.

## Risposte

**Risposta 2.1.** (a) `Class<String>`, nessuna eccezione. (b) `Class<? extends String>`, nessuna eccezione. (c) `Class<?>`, può lanciare `ClassNotFoundException` (controllata: va dichiarata o catturata). (d) `Class<? extends Object>`: il tipo statico segue quello della variabile, non dell'oggetto. Tutte e quattro danno lo stesso oggetto, confrontabile con `==`.

**Risposta 2.2.** Nell'ordine `getName`, `getSimpleName`, `getTypeName`:

- `Map.Entry`: `java.util.Map$Entry`, `Entry`, `java.util.Map$Entry`;
- `double[]`: `[D`, `double[]`, `double[]`;
- `String[][]`: `[[Ljava.lang.String;`, `String[][]`, `java.lang.String[][]`.

`forName` vuole il nome binario, il primo dei tre; `"java.util.Map.Entry"` e `"double[]"` danno `ClassNotFoundException`.

**Risposta 2.3.** (a) vero; (b) vero, una classe è assegnabile a sé stessa; (c) vero, anche un'interfaccia è assegnabile a `Object`; (d) falso, un `Rectangle` non è un `Circle`; (e) falso: `int.class` è il primitivo, non `Integer`, e i primitivi non hanno superclassi. Sarebbe vero con `Integer.class`.

**Risposta 2.4.** (a) No: dichiarare una variabile di quel tipo non tocca la classe. (b) No: `VERSIONE` è una costante copiata nel chiamante. (c) Sì, prima della lettura: `VOCI` non è una costante (è un oggetto), e leggere un campo statico inizializza. (d) No: `initialize` è `false`; il loader `null` è il bootstrap, che però non trova classi nostre, quindi in pratica `ClassNotFoundException`. (e) No: il letterale di classe non inizializza.

**Risposta 2.5.** (a) `NoClassDefFoundError`, nel punto in cui il codice compilato la nomina: la classe c'era alla compilazione, non c'è all'esecuzione. (b) `ClassNotFoundException` da `forName`, controllata, che il programma può gestire. (c) La prima volta `ExceptionInInitializerError` con la `IllegalStateException` come causa; la seconda `NoClassDefFoundError` con il messaggio "Could not initialize class": la classe è in uno stato irrecuperabile.

Capitolo 3

# Dentro la classe: campi, metodi, costruttori

Nel capitolo 2 l'oggetto `Class` ci ha detto che cosa una classe *è*: nome, gerarchia, natura. Qui gli chiediamo che cosa *contiene* e che cosa gli si può far fare: quali metodi ha, con quali parametri, e come si chiama un metodo di cui si conosce solo il nome; come si crea un'istanza senza scrivere `new`; come si legge un campo, anche privato, e fin dove la JVM lo permette. È la parte della reflection che i framework usano tutto il giorno e che costa qualcosa, in tempo e in sicurezza: la misureremo. Alla fine il menù del Plot si riempirà da solo con tutte le funzioni matematiche della libreria standard, senza che nessuna sia scritta nel sorgente: un passo in più verso la prima richiesta del capitolo 1.

## 3.1 I membri come oggetti

Come la classe, anche ciò che la classe dichiara è rappresentato da oggetti: `Field` per un campo, `Method` per un metodo, `Constructor` per un costruttore, `Parameter` per un parametro, tutti nel package `java.lang.reflect`. Si ottengono dall'oggetto `Class` con due famiglie di metodi, e la differenza fra le due è la prima cosa da imparare. Proviamole su una versione del `Circle` con membri di ogni visibilità:

```
public class Circle implements Shape2d {
    private final double radius;
    public Circle(double radius) { this.radius = radius; }
    public double getRadius() { return radius; }
    public double area() { return Math.PI * radius * radius; }
    public double perimeter() { return 2 * Math.PI * radius; }
    protected String descrizione() { return "cerchio di raggio " + radius; }
    private boolean valido() { return radius > 0; }
    @Override public String toString() { return "Circle(" + radius + ")"; }
}
```

Il programma stampa i metodi restituiti da `getMethods()` e da `getDeclaredMethods()`, ordinati per nome, con la classe che li dichiara fra parentesi quadre; `firma` è un metodo nostro di dieci righe che compone modificatori, tipo di ritorno, nome e parametri.

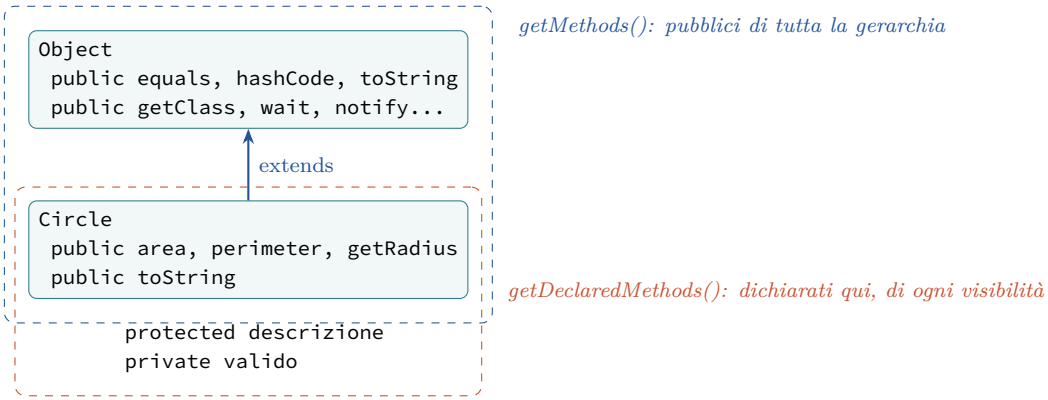
```
Arrays.stream(Circle.class.getMethods())
      .sorted(Comparator.comparing(Method::getName))
      .forEach(m -> System.out.println("  " + firma(m)));
```

```
getMethods():
  public double area()  [Circle]
  public boolean equals(Object arg0)  [Object]
  public final native Class getClass()  [Object]
  public double getRadius()  [Circle]
  public native int hashCode()  [Object]
  public final native void notify()  [Object]
  public final native void notifyAll()  [Object]
  public double perimeter()  [Circle]
  public String toString()  [Circle]
  public final void wait(long arg0)  [Object]
  public final void wait(long arg0, int arg1)  [Object]
  public final void wait()  [Object]
getDeclaredMethods():
  public double area()  [Circle]
  protected String descrizione()  [Circle]
  public double getRadius()  [Circle]
  public double perimeter()  [Circle]
  public String toString()  [Circle]
  private boolean valido()  [Circle]
```

`getMethods()` restituisce i metodi *pubblici*, compresi quelli ereditati: per questo compaiono `wait`, `notify` e `getClass` di `Object`, e non compaiono `descrizione` e `valido`. Il metodo `getDeclaredMethods()`, invece, restituisce i metodi *dichiarati in quella classe*, di qualunque visibilità, e nessuno di quelli ereditati: `equals` e `hashCode` spariscono perché `Circle` non li ridefinisce, `toString` resta perché lo ridefinisce. Le stesse due famiglie esistono per i campi, con `getFields` e `getDeclaredFields`, e per i costruttori, con `getConstructors` e `getDeclaredConstructors`; per cercare un membro preciso ci sono le versioni al singolare, che vogliono il nome e i tipi dei parametri:

```
Method area = Circle.class.getMethod("area");                // pubblico,
    ereditato
Method valido = Circle.class.getDeclaredMethod("valido");    // dichiarato,
    privato
Method charAt = String.class.getMethod("charAt", int.class); // la firma: i tipi
```

Se il metodo non c'è, `NoSuchMethodException`, controllata. La firma va data con i tipi esatti: `int.class` e non `Integer.class`, perché per la reflection, come per il compilatore, sono due tipi diversi, e `charAt(Integer)` non esiste.




Due insiemi diversi, che si sovrappongono solo sui pubblici dichiarati qui.

**Attenzione 3.1: L'ordine non è garantito**

La specifica di `getMethods()` e `getDeclaredMethods()` dice che gli elementi dell'array non sono in nessun ordine particolare, e in pratica l'ordine cambia fra versioni della JVM e perfino fra esecuzioni. Un programma che stampa i metodi "come vengono" sembra funzionare, e un giorno il menù del Plot esce mescolato. Chi ha bisogno di un ordine lo impone: lo `sorted` dell'esempio non è un vezzo.

Ogni `Method` sa tutto del metodo che rappresenta: il nome con `getName`, il tipo di ritorno con `getReturnType`, i parametri con `getParameterTypes`, `getParameterCount` e `getParameters` (nome e tipo di ciascuno), i modificatori con `getModifiers`, da leggere con `Modifier` come nel capitolo 2, la classe che lo dichiara con `getDeclaringClass`, le eccezioni dichiarate con `getExceptionTypes`. I nomi dei parametri meritano una nota: nell'output di sopra compaiono come `arg0` e `arg1`, perché il compilatore non li salva nel `.class` se non glielo si chiede con l'opzione `-parameters`; i tipi ci sono sempre, i nomi solo a richiesta. Eclipse la attiva nelle preferenze del compilatore, e un programma che vuole mostrare `f(double x)` invece di `f(double arg0)` ne ha bisogno.

## 3.2 Invocare

Un `Method` non si limita a descrivere: si può *chiamare*, con `invoke`, passando l'oggetto su cui chiamarlo e gli argomenti. Il risultato è un `Object`, qualunque sia il tipo di ritorno dichiarato:

```
Method sin = Math.class.getMethod("sin", double.class);
Object r = sin.invoke(null, Math.PI / 2);            // static: bersaglio null
System.out.println(r + " " + r.getClass().getSimpleName());

Method charAt = String.class.getMethod("charAt", int.class);
System.out.println(charAt.invoke("reflection", 2));
```

```
1.0 Double
f
```

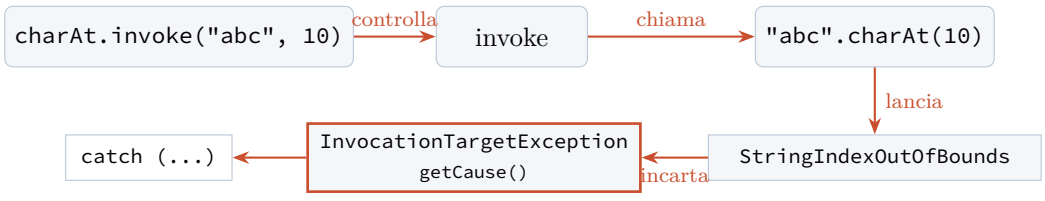
Tre cose da notare. Per un metodo statico il bersaglio è `null`: non c'è un oggetto su cui chiamarlo. I primitivi viaggiano *incartati*: `Math.PI / 2` diventa un `Double` all'andata, perché `invoke` accetta `Object...`, e il `double` restituito torna come `Double`, da cui un cast a `double`

(con unboxing) quando serve il numero. E il compilatore non controlla niente: non sa che `charAt` vuole un `int`, né che restituisce un `char`. Tutti i controlli che faceva lui li fa `invoke`, a runtime, e li segnala con le eccezioni:

```
try {
    charAt.invoke("abc", 10);
} catch (InvocationTargetException e) {
    System.out.println("incartata: " + e.getCause());
}
try {
    charAt.invoke("abc", "due");
} catch (IllegalArgumentException e) {
    System.out.println("argomenti: " + e.getMessage());
}
try {
    charAt.invoke(42, 0);
} catch (IllegalArgumentException e) {
    System.out.println("bersaglio: " + e.getMessage());
}
```

```
incartata: java.lang.StringIndexOutOfBoundsException: Index 10 out of bounds
  for length 3
argomenti: argument type mismatch
bersaglio: object of type java.lang.Integer is not an instance of java.lang.String
```

La prima eccezione è la più importante. Il metodo chiamato ha lanciato un errore *suo*, la `StringIndexOutOfBoundsException`; ma a noi arriva `InvocationTargetException`, che la contiene come causa. `invoke` distingue così i propri fallimenti (non ho potuto chiamare il metodo: argomenti sbagliati, bersaglio sbagliato, metodo inaccessibile) da quelli del metodo (l'ho chiamato, e ha lanciato): i primi sono `IllegalArgumentException` e `IllegalAccessException`, il secondo è sempre `InvocationTargetException`, e l'eccezione vera si recupera con `getCause()`. Chi cattura solo `Exception` e stampa il messaggio vede "null", perché `InvocationTargetException` non ha un messaggio proprio, e passa un'ora a cercare un errore che sta un livello più sotto.




Un'eccezione del metodo chiamato arriva sempre incartata; i fallimenti di `invoke` no.

**Attenzione 3.2: Varargs: un array passato come argomento unico**

`invoke(Object obj, Object... args)` è un metodo a argomenti variabili, e il compilatore applica la regola di sempre: un `Object[]` passato da solo *è* l'array degli argomenti, non un argomento. Per chiamare `Arrays.asList(T...)`, che vuole un solo parametro di tipo array, la riga ingenua fallisce:

```
Method asList = Arrays.class.getMethod("asList", Object[].class);
Object[] parole = { "a", "b" };
asList.invoke(null, parole);           // wrong number of arguments: 2
    expected: 1
asList.invoke(null, (Object) parole);          // giusto: un argomento, un
    array
asList.invoke(null, new Object[] { parole });  // equivalente, pi\‘u
    esplicito
```

Lo stesso tranello aspetta `getMethod("f", tipi)` quando i tipi stanno già in un `Class<?>[]`, e in quel caso è la forma giusta, non un errore.

Un `Method` e una lambda sono parenti stretti: entrambi rappresentano "una cosa da chiamare", e passare dall'uno all'altra è il modo di consegnare al resto del programma, che parla il linguaggio delle interfacce funzionali, ciò che la reflection ha trovato. Il Plot lo farà nella sezione 3.6; il nucleo è questo:

```
DoubleUnaryOperator f = x -> {
    try {
        return (double) sin.invoke(null, x);
    } catch (ReflectiveOperationException e) {
        throw new IllegalStateException(e);
    }
};
```

Le eccezioni controllate di `invoke` non possono uscire da una lambda che implementa `applyAsDouble`, che non le dichiara: si incartano in una non controllata, scegliendo quella che dice il vero (`IllegalStateException`: "questo metodo doveva essere chiamabile, e non lo è"). È lo schema che useremo ogni volta.

## 3.3 Costruire

Un `Constructor<T>` si ottiene con `getConstructor` (pubblici) o `getDeclaredConstructor` (tutti), passando i tipi dei parametri, e si chiama con `newInstance`, che restituisce direttamente un `T`:

```
Constructor<Circle> c = Circle.class.getConstructor(double.class);
Circle cerchio = c.newInstance(2.0);
Class<?> k = Class.forName("membri.Circle");
Object o = k.getDeclaredConstructor(double.class).newInstance(1.0);
```

La prima riga dà un `Circle` tipato, perché `Circle.class` è un `Class<Circle>`; la terza dà un `Object`, perché `forName` dà un `Class<?>`, ed è la forma del Plot del capitolo 2. Se il costruttore con quella firma non esiste, `NoSuchMethodException`: una classe con il solo costruttore

`Circle(double)` non ha quello senza argomenti, e un plugin che lo dimentica si scopre così. Le eccezioni di `newInstance` sono le stesse di `invoke`, con `InvocationTargetException` per un'eccezione lanciata dal costruttore, più `InstantiationException` per una classe astratta o un'interfaccia.

**Attenzione 3.3: `Class.newInstance()` è deprecato, e per un buon motivo**

Esiste ancora `classe.newInstance()`, senza passare dal costruttore, e il codice scritto prima di Java 9 lo usa dappertutto. È deprecato perché fa una cosa che nessun altro metodo fa: se il costruttore lancia un'eccezione *controllata*, la rilancia così com'è, senza incartarla e senza che il compilatore ne sappia nulla, quindi un `catch` scritto bene non la cattura. La forma giusta è `classe.getDeclaredConstructor().newInstance()`: una chiamata in più, e le eccezioni dove ci si aspetta di trovarle.

I costruttori privati sono privati per il compilatore, non per la reflection. Un singleton con il costruttore privato e un'istanza unica statica si viola in tre righe:

```
Registro a = Registro.getInstance();
Constructor<Registro> privato = Registro.class.getDeclaredConstructor();
privato.setAccessible(true);
Registro b = privato.newInstance();
System.out.println("stesso oggetto? " + (a == b));     // false
```

`setAccessible(true)` chiede di saltare il controllo di visibilità; senza, la chiamata fallisce con `IllegalAccessException`. Funziona perché `Registro` è una classe nostra, nello stesso modulo (quello senza nome, in cui vive il codice che non dichiara moduli): la sezione seguente dice dove si ferma.

## 3.4 Leggere e scrivere i campi

Un `Field` si legge con `get(oggetto)` e si scrive con `set(oggetto, valore)`, più le varianti per i primitivi (`getDouble`, `setInt`) che evitano l'incartamento; per un campo statico l'oggetto è `null`. La visibilità si applica come per i metodi, e `setAccessible` la scavalca:

```
Circle c = new Circle(3);
Field radius = Circle.class.getDeclaredField("radius");
try {
    radius.get(c);
} catch (IllegalAccessException e) {
    System.out.println("privato: " + e.getMessage());
}
radius.setAccessible(true);
System.out.println("radius = " + radius.get(c) + " / " + radius.getDouble(c));
radius.set(c, 10.0);                                    // anche se final
System.out.println(c + " area " + c.area());
System.out.println(Math.class.getField("PI").get(null));
```

```
privato: class membri.Campi cannot access a member of class membri.Circle
  with private access
radius = 3.0, come double: 3.0
Circle(10.0) area 314.1592653589793
3.141592653589793
```

Il campo è `private final`, e la reflection lo ha letto e perfino riscritto. Sulla scrittura di un campo `final` vale un avviso: la JVM più recente la segnala sulla console e annuncia che in futuro la bloccherà, perché rompe garanzie su cui il JIT e i thread fanno affidamento (un `final` letto una volta può essere stato copiato in un registro, e la scrittura riflessiva non lo aggiorna). Qui conta come dimostrazione di quanto in là si può andare, non come tecnica.

Quanto in là, però, non è ovunque. Proviamo la stessa cosa su un campo privato di `String`:

```
Field value = String.class.getDeclaredField("value");
value.setAccessible(true);
```

```
java.lang.reflect.InaccessibleObjectException: Unable to make field private final
  byte[] java.lang.String.value accessible: module java.base does not "opens
  java.lang" to unnamed module
```

Dal Java 16 JAVA 16 la libreria standard è *incapsulata fortemente*: le classi del modulo `java.base` e degli altri moduli del JDK non aprono i loro membri privati alla reflection, e `setAccessible` fallisce con `InaccessibleObjectException`, non controllata. Prima si poteva, e molte librerie lo facevano, con il risultato che ogni cambiamento interno del JDK le rompeva; il sistema dei moduli ha chiuso la porta. L'opzione di avvio `-add-opens java.base/java.lang=ALL-UNNAMED` la riapre per un package, ed è una toppa per far girare codice vecchio, non un modo di scrivere codice nuovo. Le classi nostre, e quelle delle librerie che non dichiarano moduli, restano aperte: è per questo che `radius` si è lasciato leggere.

**Osservazione 3.1**

`setAccessible` rompe l'incapsulamento, cioè la promessa su cui l'autore di una classe ha costruito la sua correttezza. È legittimo in tre casi: uno strumento generale che deve vedere tutto (un serializzatore, un `toString` per qualunque oggetto, un debugger), un framework di test che deve raggiungere lo stato interno, una libreria che inietta dipendenze in campi privati per scelta di chi li ha dichiarati. Non lo è mai come scorciatoia per usare un metodo che l'autore ha reso privato: se serve, si cambia la classe o si chiede un'interfaccia; se la classe è di altri, il privato può cambiare alla prossima versione senza avviso, e la scorciatoia smette di funzionare quando meno lo si aspetta.

## 3.5 Il costo

La reflection costa, e vale la pena sapere quanto, per decidere dove usarla. Il programma seguente chiama venti milioni di volte `Math.abs`, un metodo così economico che il tempo misurato è quasi tutto nel meccanismo di chiamata, in quattro modi: diretta, attraverso una lambda `Math::abs`, attraverso `invoke` su un `Method` cercato una volta, e cercando il `Method` con `getMethod` a ogni chiamata (due milioni di volte, perché i venti sarebbero troppi).

```
giro 1: diretta    12 ms, lambda    12 ms, invoke   111 ms, invoke+lookup   430 ms
giro 2: diretta    12 ms, lambda    12 ms, invoke    81 ms, invoke+lookup   408 ms
giro 3: diretta    11 ms, lambda    11 ms, invoke    78 ms, invoke+lookup   407 ms
```

| chiamata | per chiamata | rispetto alla diretta |
|---|---:|---:|
| diretta, `Math.abs(x)` | 0,6 ns | 1 |
| lambda, `Math::abs` | 0,6 ns | 1 |
| `invoke` su un `Method` noto | 4 ns | 7 |
| `getMethod` + `invoke` | 200 ns | 350 |

La lambda non costa nulla in più della chiamata diretta: il JIT la riconosce e la sostituisce con il codice del metodo (*inlining*). `invoke` costa alcuni nanosecondi: deve controllare il bersaglio, gli argomenti e l'accesso a ogni chiamata, e il JIT lo ottimizza solo in parte. La ricerca del metodo è un altro ordine di grandezza: `getMethod` scorre i metodi della classe e delle superclassi confrontando nomi e firme, e crea un oggetto `Method` nuovo ogni volta. Da qui la regola.

**Definizione 3.1: Ispeziona una volta, chiama mille**

La ricerca per reflection (`getMethod`, `getDeclaredField`, `getConstructor`) si fa una volta, quando si scopre che cosa c'è, e il risultato si conserva in un campo, in una mappa, o si incarta in una lambda; la chiamata (`invoke`, `get`, `newInstance`) si può ripetere. Un `getMethod` dentro un ciclo è il primo errore di prestazioni della reflection, e il più facile da evitare.

Per il Plot i numeri dicono che non c'è di che preoccuparsi: un ridisegno campiona la funzione una volta per pixel, circa mille chiamate, e mille `invoke` sono quattro microsecondi. In un ciclo da un miliardo di iterazioni la differenza fra 0,6 e 4 nanosecondi sono tre secondi e mezzo, e allora conviene uscire dalla reflection prima del ciclo. Il capitolo 8 accenna agli strumenti con cui si fa (i *method handle*), che portano una chiamata scoperta a runtime al costo di una diretta.

## 3.6 L'esempio guida: il menù si riempie da solo

Il Plot del capitolo 2 sa caricare una classe dal nome; ma l'utente, prima di scrivere una funzione sua, vorrebbe trovare nel menù almeno quelle che Java conosce già. La classe `Math` ne ha ventisei della forma `static double f(double)`: seno, coseno, esponenziale, logaritmo, radice… Invece di elencarle nel sorgente, chiediamole alla classe. La logica sta in una classe `Catalogo`, che tiene una mappa ordinata da un nome a una `Funzione` e sa riempirla in due modi:

```
public class Catalogo {

    private final Map<String, Funzione> funzioni = new LinkedHashMap<>();

    public Catalogo() {
        aggiungiMetodiDi(Math.class);
        for (String nome : new String[] { "Gaussiana", "Sinc", "Onda" })
            aggiungiClasse("it.unibs.pajc.plot.funzioni." + nome);
    }

    /** Tutti i metodi "public static double f(double)" di una classe. */
    public void aggiungiMetodiDi(Class<?> classe) {
        Arrays.stream(classe.getMethods())
              .filter(m -> Modifier.isStatic(m.getModifiers()))
              .filter(m -> m.getReturnType() == double.class)
              .filter(m -> m.getParameterCount() == 1)
              .filter(m -> m.getParameterTypes()[0] == double.class)
              .sorted(Comparator.comparing(Method::getName))
              .forEach(m -> funzioni.put(m.getName() + "(x)", incarta(m)));
    }

    /** Una classe Funzione dal suo nome, istanziata per reflection. */
    public void aggiungiClasse(String nomeClasse) {
        try {
            Class<?> classe = Class.forName(nomeClasse);
            if (!Funzione.class.isAssignableFrom(classe))
                throw new IllegalArgumentException(nomeClasse + ": non Funzione");
            Funzione fn = (Funzione)
    classe.getDeclaredConstructor().newInstance();
            funzioni.put(classe.getSimpleName(), fn);
        } catch (ReflectiveOperationException e) {
            throw new IllegalArgumentException("inutilizzabile: " + nomeClasse,
    e);
        }
    }

    /** Da un Method statico a una Funzione: lookup una volta, invoke a ogni uso.
    */
    private static Funzione incarta(Method m) {
        return x -> {
            try {
                return (double) m.invoke(null, x);
            } catch (ReflectiveOperationException e) {
                throw new IllegalStateException("errore in " + m.getName(), e);
            }
        };
    }

    public Map<String, Funzione> funzioni() {
        return funzioni;
    }
}
```

`aggiungiMetodiDi` è la sezione 3.1 messa al lavoro: `getMethods()` dà tutti i metodi pubblici di `Math`, i tre filtri tengono quelli statici, che restituiscono `double` e che ricevono un solo `double` (il confronto è con `==`, perché gli oggetti `Class` sono unici), e `sorted` impone l'ordine che la reflection non promette. `incarta` è la sezione 3.2: il `Method` resta catturato nella lambda, cercato una volta e chiamato a ogni pixel, secondo la regola della sezione 3.5. `aggiungiClasse` è la ricetta del capitolo 2, con le eccezioni tradotte in una `IllegalArgumentException` che dice quale classe non è andata bene. Si noti che `Funzione` è un'interfaccia funzionale, e quindi la lambda di `incarta` la implementa direttamente: metodi di `Math` e classi nostre finiscono nella stessa mappa, con lo stesso tipo, e il resto del programma non distingue gli uni dalle altre.

La finestra torna ad avere il menù, costruito dalle chiavi della mappa, e una riga di stato che misura quanto costa un ridisegno:

```
private final Catalogo catalogo = new Catalogo();
private final JComboBox<String> cmbFunzione =
        new JComboBox<>(catalogo.funzioni().keySet().toArray(new String[0]));

private void disegna() {
    String nome = (String) cmbFunzione.getSelectedItem();
    Funzione fn = catalogo.funzioni().get(nome);
    long t = System.nanoTime();
    pnlPlot.setFunzione(fn::f);
    pnlPlot.paintImmediately(0, 0, pnlPlot.getWidth(), pnlPlot.getHeight());
    long us = (System.nanoTime() - t) / 1000;
    lblStato.setText(nome + ": " + catalogo.funzioni().size() + " funzioni, "
            + "ridisegno in " + us + " us");
}
```

Il menù mostra `abs(x)`, `acos(x)`, `asin(x)`, ..., `ulp(x)`, poi `Gaussiana`, `Sinc` e `Onda`: ventinove voci, e nel sorgente del Plot non ne è scritta nessuna. Il ridisegno di una funzione di `Math` attraverso `invoke` costa qualche millisecondo, quasi tutto nel disegno, e non si distingue da uno fatto con la lambda diretta. Ciò che manca è ancora la funzione *dell'utente*: per ora il programma sa guardare solo dentro classi che ha già. I capitoli 5 e 6 aprono le due porte verso l'esterno; prima, nel capitolo 4, diamo alle funzioni un modo di presentarsi con un nome e un intervallo.

**Ricetta: chiamare un metodo dal nome**

1. cercarlo una volta: `getMethod(nome, tipi...)` per un pubblico anche ereditato, `getDeclaredMethod` per uno dichiarato nella classe (`NoSuchMethodException` se non c'è); conservare il `Method`;
2. se non è pubblico, `setAccessible(true)`, solo su classi nostre e solo con un motivo;
3. `invoke(bersaglio, argomenti...)`: `null` come bersaglio se è statico; attenzione a un array passato da solo;
4. `catch (InvocationTargetException e)`: l'errore del metodo è in `e.getCause()`; `IllegalArgumentException` e `IllegalAccessException` sono errori nostri;
5. il risultato è un `Object`: cast al tipo atteso, con l'unboxing per i primitivi.

## Esercizi

### Esercizio 3.1: Quanti metodi (⋆)

Per la classe `Circle` di questo capitolo, dire senza eseguire quanti elementi hanno `getMethods()` e `getDeclaredMethods()`, e che cosa cambia nei due numeri se si aggiunge a `Circle` un metodo `public boolean equals(Object o)`.

### Esercizio 3.2: La firma giusta (⋆)

Quali di queste chiamate lanciano `NoSuchMethodException`? (a) `String.class.getMethod("length");` (b) `String.class.getMethod("charAt", Integer.class);` (c) `String.class.getMethod("substring", int.class, int.class);` (d) `String.class.getMethod("valueOf", Object.class);` (e) `Circle.class.getMethod("valido");` (f) `Circle.class.getMethod("hashCode").`

### Esercizio 3.3: Chi lancia (⋆⋆)

Per ciascuna riga dire quale eccezione arriva al chiamante, e, se è `InvocationTargetException`, quale eccezione è la causa:

```
Method parse = Integer.class.getMethod("parseInt", String.class);
parse.invoke(null, "42");          // (a)
parse.invoke(null, "quarantadue"); // (b)
parse.invoke(null, 42);            // (c)
parse.invoke("x", "42");           // (d)
parse.invoke(null);                // (e)
```

### Esercizio 3.4: Il singleton violato (⋆⋆)

Nel frammento della sezione 3.3 si ottengono due `Registro`. Che cosa succede se si toglie la riga con `setAccessible`? E se `Registro` fosse una classe della libreria standard, per esempio `java.lang.Runtime`, che ha anch'essa il costruttore privato? Come potrebbe l'autore di `Registro` difendersi dalla reflection? (Un suggerimento: che cosa fa il costruttore se `UNICO` è già diverso da `null`?)

### Esercizio 3.5: Dentro o fuori dal ciclo (⋆⋆)

Un programma calcola la somma di `f(i)` per `i` da 1 a un miliardo, dove `f` è un metodo statico scelto dal nome a runtime. Stimare con la tabella della sezione 3.5 il tempo in più rispetto alla chiamata diretta (a) con `getMethod` dentro il ciclo, (b) con il `Method` cercato una volta e `invoke` nel ciclo. Come si scrive la versione (b) in modo che il ciclo non contenga nessun `try`?

*I problemi che seguono chiedono un programma completo, in ordine crescente di difficoltà. Le soluzioni sono nel repository delle soluzioni del corso, nel progetto e nel package indicati.*

### Esercizio 3.6: Un `toString` per tutti (⋆)

Scrivere `static String descrivi(Object o)` che restituisce una stringa come `Circle[radius=3.0]` o `Persona[nome=Ada, anni=36, indirizzo=Indirizzo[via=..., civico=12]]`: il nome semplice della classe e tutti i campi d'istanza, compresi quelli privati e quelli ereditati dalle superclassi, con i valori. I campi statici si saltano; un campo il cui valore è un oggetto di una classe nostra si descrive ricorsivamente, uno di una classe del JDK (`String`, `Integer`, `List`...) con il suo `toString`. Provarlo su tre classi diverse, una con una superclasse.

Soluzione: progetto `sol-dxtr-01-c03`, package `it.unibs.pajc.redolfi.e06`.

### Attenzione 3.4: I campi ereditati non sono dichiarati

`getDeclaredFields()` dà solo i campi dichiarati in quella classe, e `getFields()` solo quelli pubblici: per avere i privati della superclasse bisogna risalire con `getSuperclass()` e ripetere, fermandosi prima di `Object`. E un oggetto che contiene un riferimento a sé stesso manda la ricorsione in `StackOverflowError`: si tiene traccia di che cosa si è già visitato.

### Esercizio 3.7: Uguali campo per campo (⋆⋆)

Scrivere `static boolean stessiCampi(Object a, Object b)`: vero se i due oggetti hanno la stessa classe e tutti i campi d'istanza uguali, confrontando i primitivi per valore, gli oggetti con `equals`, e ricorsivamente le classi nostre che non ridefiniscono `equals`. Decidere, e documentare, che cosa fare dei campi `static` e `transient`. Mostrare un caso in cui `stessiCampi` e `equals` danno risposte diverse e spiegare quale ha ragione.

Soluzione: progetto `sol-dxtr-01-c03`, package `it.unibs.pajc.redolfi.e07`.

### Esercizio 3.8: Una shell per gli oggetti (⋆⋆)

Scrivere un programma a riga di comando che tiene una mappa da nomi a oggetti (all'inizio: `s` = una `String`, `c` = un `Circle`, `l` = una `ArrayList<String>`) e accetta righe come `s.length()`, `s.substring(2, 5)`, `l.add(ciao)`, `c.area()`: il nome dell'oggetto, un punto, il metodo e gli argomenti fra parentesi, separati da virgole. Il programma cerca fra i metodi pubblici uno con quel nome e quel numero di parametri, converte ogni argomento nel tipo del parametro (`int`, `double`, `boolean`, `String`), lo invoca e stampa il risultato, o un messaggio sensato per ogni cosa che può andare storta. `fine` esce.

Soluzione: progetto `sol-dxtr-01-c03`, package `it.unibs.pajc.redolfi.e08`.

**Esercizio 3.9: Copiare le proprietà (⋆⋆⋆)**

Molte librerie trasportano dati fra oggetti di classi diverse accoppiando i *getter* dell'uno ai *setter* dell'altro: `getNome()` di `Persona` va in `setNome(String)` di `PersonaDto` se nome e tipo coincidono. Scrivere `static int copia(Object sorgente, Object destinazione)` che fa questo per tutte le coppie compatibili, restituisce quante proprietà ha copiato, ignora le altre, e tratta anche i getter `isX()` dei `boolean` e i record (i cui "getter" si chiamano come il componente). Provarlo con due classi che condividono alcune proprietà e ne hanno altre proprie.
*Suggerimento: costruire prima una mappa nome → (tipo, `Method`) dei getter della sorgente e una dei setter della destinazione, poi intersecarle.*

Soluzione: progetto `sol-dxtr-01-c03`, package `it.unibs.pajc.redolfi.e09`.

## Risposte

**Risposta 3.1.** `getMethods()`: 12 (quattro pubblici di `Circle`: `area`, `perimeter`, `getRadius`, `toString`, più otto di `Object`: `equals`, `hashCode`, `getClass`, `notify`, `notifyAll` e tre `wait`). `getDeclaredMethods()`: 6. Con un `equals` in `Circle` il primo numero resta 12 (quello di `Circle` sostituisce quello di `Object`, non si aggiunge), il secondo diventa 7.

**Risposta 3.2.** (b): `charAt` vuole `int`, e `Integer.class` è un altro tipo. (e): `valido` è privato, e `getMethod` vede solo i pubblici (con `getDeclaredMethod` si troverebbe). Le altre vanno: (f) trova `hashCode` ereditato da `Object`, (d) trova `valueOf(Object)`, che esiste accanto alle versioni per i primitivi.

**Risposta 3.3.** (a) nessuna, restituisce l'`Integer` 42. (b) `InvocationTargetException` con causa `NumberFormatException`: è `parseInt` a lanciare. (c) `IllegalArgumentException` ("argument type mismatch"): l'argomento è un `Integer`, il parametro una `String`; il metodo non viene chiamato. (d) nessuna: per un metodo statico il bersaglio viene ignorato, anche se non è `null`. (e) `IllegalArgumentException` ("wrong number of arguments").

**Risposta 3.4.** Tre risposte.

- Senza `setAccessible`: `newInstance` lancia una `IllegalAccessException`, e il secondo oggetto non nasce.
- Con `java.lang.Runtime` si fermerebbe prima: `setAccessible` lancerebbe `InaccessibleObjectException`, perché il modulo `java.base` non apre i suoi privati.
- La difesa: il costruttore lancia un'eccezione se `UNICO` non è `null` (la seconda costruzione fallisce, e l'eccezione arriva incartata in una `InvocationTargetException`); oppure un `enum` con una sola costante, che la reflection si rifiuta di istanziare.

**Risposta 3.5.** (a) Circa 200 ns per iterazione: duecento secondi in più. (b) Circa 3,5 ns per iterazione: tre secondi e mezzo. Per togliere il `try` dal ciclo si incarta il `Method` in un `DoubleUnaryOperator` prima del ciclo, come in `incarta` del catalogo: il `try` sta nella lambda, il ciclo chiama `applyAsDouble`.

Capitolo 4

# Annotazioni e proxy: come fanno i framework

Con i capitoli 2 e 3 il programma sa guardare dentro una classe e usarne i membri. Ma ciò che vede è solo ciò che il linguaggio dichiara: nomi, tipi, modificatori. Spesso serve di più: sapere che un metodo è un test, che un campo non può essere nullo, che una funzione del Plot vuole essere mostrata con un certo nome e su un certo intervallo. Le *annotazioni* sono il modo di scrivere nel codice queste informazioni in più, e la reflection è il modo di leggerle. Nella seconda parte del capitolo aggiungiamo l'ultimo attrezzo della cassetta: i *proxy dinamici*, oggetti senza una classe scritta da noi che implementano un'interfaccia intercettando ogni chiamata. Annotazioni e proxy insieme sono il meccanismo con cui JUnit trova i test, Spring collega i componenti, Hibernate salva gli oggetti: alla fine del capitolo non saranno più magia. Le funzioni del Plot, intanto, imparano a presentarsi.

## 4.1 Metadati nel codice

Le annotazioni le usiamo dal primo giorno. `@Override` dice al compilatore che il metodo sotto ridefinisce un metodo della superclasse, e il compilatore protesta se non è vero; `@FunctionalInterface` chiede di verificare che l'interfaccia abbia un solo metodo astratto; `@Deprecated` fa comparire un avviso in chi usa il membro; `@SuppressWarnings("unchecked")` spegne un avviso. In tutti e quattro i casi l'annotazione *non cambia il programma*: è un'informazione attaccata a un elemento del codice, e c'è qualcuno, il compilatore, che la legge e fa qualcosa. Senza quel qualcuno, l'annotazione è un commento con una sintassi.

**Definizione 4.1: Annotazione**

Un'annotazione è un metadato strutturato, con un tipo e con eventuali valori (*elementi*), attaccato a una dichiarazione: una classe, un metodo, un campo, un parametro, una variabile locale, un package, e dal Java 8 anche un uso di tipo. Di per sé non ha effetto; lo ha quando uno strumento la legge: il compilatore, un processore di annotazioni durante la compilazione, o un programma a tempo di esecuzione attraverso la reflection.

Questo capitolo si occupa del terzo lettore, il programma stesso. Per il Plot vogliamo poter scrivere `@Plot(nome = "Gaussiana", xMin = -4, xMax = 4)` sopra una funzione, e che il menù mostri quel nome e apra quell'intervallo: tre informazioni che non stanno in nessuna firma, e che sarebbe scomodo mettere in un file a parte da tenere allineato con il codice.

## 4.2 Dichiararne una

Un'annotazione si dichiara come un'interfaccia con una chiocciola davanti, e i suoi elementi come metodi senza parametri, con un eventuale valore di default:

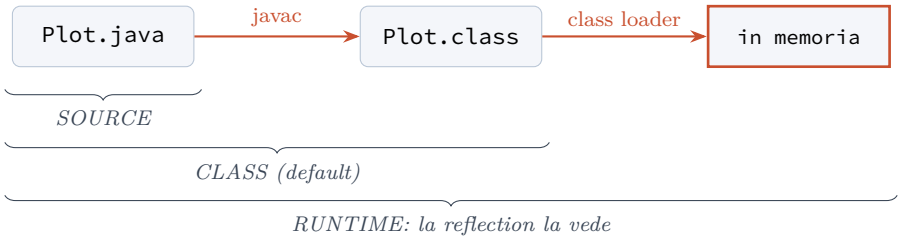
```
@Retention(RetentionPolicy.RUNTIME)
@Target({ ElementType.TYPE, ElementType.METHOD })
public @interface Plot {
    String nome();
    double xMin() default -10;
    double xMax() default 10;
}
```

`@interface` introduce il tipo; `Plot` è il nome con cui la si scriverà (`@Plot(...)`); `nome`, `xMin` e `xMax` sono i suoi elementi: chi la usa deve dare `nome`, perché non ha default, e può omettere gli altri due. Un elemento che si chiama `value` ha un privilegio: se è l'unico che si specifica, il suo nome si può omettere, e `@SuppressWarnings("unchecked")` è in realtà `@SuppressWarnings(value = "unchecked")`. I tipi ammessi per gli elementi sono pochi e fissi: primitivi, `String`, `Class`, enum, altre annotazioni, e array di questi. Niente oggetti qualunque, niente `null`: un'annotazione deve potersi scrivere come costante nel `.class`, e il compilatore rifiuta tutto ciò che non lo è.

Le due annotazioni sopra la dichiarazione sono *meta-annotazioni*, annotazioni sulle annotazioni, e la prima è quella che decide se tutto il capitolo funziona.

**Definizione 4.2: Retention: fin dove sopravvive un'annotazione**

`@Retention` dice fino a quale fase l'annotazione esiste. `RetentionPolicy.SOURCE`: solo nel sorgente, il compilatore la legge e la scarta (`@Override`, `@SuppressWarnings`). `CLASS`: scritta nel `.class`, ma la JVM non la carica, e la reflection non la vede; serve agli strumenti che leggono i file compilati, ed è il **default**. `RUNTIME`: nel `.class` e caricata in memoria, leggibile con la reflection. Per un programma che legge le proprie annotazioni serve `RUNTIME`, sempre.




I tre livelli di retention: fin dove un'annotazione arriva.

`@Target` elenca i tipi di dichiarazione su cui l'annotazione è ammessa (`TYPE` per classi, interfacce ed enum, `METHOD`, `FIELD`, `PARAMETER`, `CONSTRUCTOR`, `LOCAL_VARIABLE`, `PACKAGE`, `TYPE_USE`...); senza, vale quasi ovunque, e il compilatore non può segnalare un `@Plot` finito per sbaglio su un campo. Tre meta-annotazioni meno frequenti completano il quadro: `@Documented` fa comparire l'annotazione nella Javadoc di ciò che annota; `@Inherited` fa sì che un'annotazione su una classe valga anche per le sottoclassi (solo per le classi: i metodi non la ereditano, e le interfacce non la trasmettono); `@Repeatable` JAVA 8 permette di scrivere la stessa annotazione più volte sullo stesso elemento, dichiarando un'annotazione contenitore che le raccoglie.

## 4.3 Leggerle

Classi, metodi, campi, costruttori e parametri implementano tutti `AnnotatedElement`, con tre metodi: `isAnnotationPresent(Tipo.class)`, `getAnnotation(Tipo.class)`, che restituisce l'annotazione o `null`, e `getAnnotations()`, l'array di tutte. L'annotazione letta è un oggetto del tipo dichiarato, e i suoi elementi si leggono chiamando i metodi omonimi. Il programma seguente scorre i metodi di una classe di funzioni e stampa quelli annotati:

```
public class Funzioni {
    @Plot(nome = "gaussiana", xMin = -4, xMax = 4)
    public static double gaussiana(double x) { return Math.exp(-x * x / 2); }

    @Plot(nome = "sinc")
    public static double sinc(double x) { return Math.sin(x) / x; }

    public static double aiuto(double x) { return 0; }          // senza
    annotazione
}

for (Method m : Funzioni.class.getDeclaredMethods()) {
    Plot p = m.getAnnotation(Plot.class);
    if (p == null)
        continue;
    System.out.printf("%-12s %-10s [%5.1f, %5.1f]%n",
                      p.nome(), m.getName(), p.xMin(), p.xMax());
}
```

```
gaussiana    gaussiana  [ -4,0,   4,0]
sinc         sinc       [-10,0,  10,0]
```

`sinc` non dichiara l'intervallo e riceve i default; `aiuto` non ha l'annotazione e `getAnnotation` restituisce `null`. Si noti che il tipo restituito è `Plot`, e `p.nome()` è una chiamata normale, controllata dal compilatore: l'oggetto è in realtà un proxy (sezione 4.5) che la JVM costruisce per implementare l'interfaccia `Plot`, ma dall'esterno non si vede. Per le annotazioni ripetibili c'è `getAnnotationsByType(Tipo.class)`, che restituisce l'array anche quando l'annotazione è una sola, e che è la forma da preferire quando `@Repeatable` è in gioco.

**Attenzione 4.1: L'annotazione che non c'è**

Questo è l'errore più frequente, e il più silenzioso, di tutto il capitolo.

```
@interface Nota { String value(); }              // retention: CLASS, il
    default

@Nota("non la vedrete")
class Annotata { }

System.out.println(Annotata.class.getAnnotation(Nota.class));     // null
```

Tutto compila, l'annotazione è scritta, è perfino nel `.class`, e `getAnnotation` restituisce `null` senza nessuna eccezione, come se l'annotazione non fosse mai stata scritta. Un runner di test che "non trova nessun test" e una funzione che "non compare nel menù" quasi sempre hanno questa causa: manca `@Retention(RetentionPolicy.RUNTIME)`. Da scrivere per prima cosa, prima ancora degli elementi.

**Osservazione 4.1**

Un'annotazione non fa niente. `@Test` non esegue il metodo, `@Plot` non lo mette nel menù, `@NonNull` non impedisce di passare `null`: ciascuna è una promessa che qualcun altro deve mantenere, e quel qualcuno è un programma, scritto da qualcuno, che cerca l'annotazione con la reflection e agisce. Capire questo toglie metà della magia ai framework: quando si scrive `@Autowired` o `@Entity`, da qualche parte c'è un ciclo su `getDeclaredFields()` con un `isAnnotationPresent` dentro. L'altra metà sono i proxy, fra poco.

## 4.4 Un mini-JUnit in quaranta righe

Il modo migliore di capire un framework è scriverne uno piccolo. Un *test* è un metodo pubblico senza parametri che, chiamato, o termina normalmente (passa) o lancia un'eccezione (fallisce); alcune eccezioni sono attese, e in quel caso il test passa solo se arrivano. Servono due annotazioni, una per i test, con l'eccezione attesa come elemento opzionale, e una per un metodo da eseguire prima di ciascuno:

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Test {
    Class<? extends Throwable> atteso() default Nessuna.class;
    final class Nessuna extends Throwable { }
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Prima { }
```

Il valore di default di un elemento non può essere `null`: per dire "nessuna eccezione attesa" si usa una classe apposta, `Nessuna`, che nessun metodo lancerà mai. È lo stesso trucco di JUnit

4 (`expected = None.class`). Il runner cerca i metodi annotati, li ordina, e per ciascuno crea un'istanza nuova della classe, esegue i `@Prima`, chiama il test e interpreta l'esito:

```
public static void esegui(Class<?> classe) throws ReflectiveOperationException {
    List<Method> test = new ArrayList<>(), prima = new ArrayList<>();
    for (Method m : classe.getDeclaredMethods()) {
        if (m.isAnnotationPresent(Test.class)) test.add(m);
        if (m.isAnnotationPresent(Prima.class)) prima.add(m);
    }
    test.sort(Comparator.comparing(Method::getName));
    int passati = 0;
    for (Method m : test) {
        Object istanza = classe.getDeclaredConstructor().newInstance();
        for (Method p : prima) p.invoke(istanza);
        Class<? extends Throwable> atteso = m.getAnnotation(Test.class).atteso();
        String esito;
        try {
            m.invoke(istanza);
            esito = atteso == Test.Nessuna.class ? "ok"
                    : "FALLITO: attesa " + atteso.getSimpleName();
        } catch (InvocationTargetException e) {
            Throwable t = e.getCause();
            esito = atteso.isInstance(t)
                    ? "ok (" + t.getClass().getSimpleName() + ")"
                    : "FALLITO: " + t;
        }
        if (esito.startsWith("ok")) passati++;
        System.out.printf("  %-24s %s%n", m.getName(), esito);
    }
    System.out.printf("%s: %d/%d passati%n", classe.getSimpleName(), passati,
                      test.size());
}
```

Tutto il capitolo 3 è qui. I metodi si trovano con `getDeclaredMethods`; l'istanza si crea con il costruttore (`getDeclaredConstructor`, poi `newInstance`); il test si chiama con `invoke`; l'eccezione che il test lancia arriva incartata in una `InvocationTargetException` e si legge con `getCause()`; infine `isInstance`, del capitolo 2, confronta l'eccezione arrivata con quella attesa, accettando anche le sottoclassi come farebbe un `catch`. Una classe di prova con uno stack:

```java
public class ProvaStack {
    private Deque<Integer> stack;

    @Prima
    public void prepara() {
        stack = new ArrayDeque<>();
        stack.push(1);
        stack.push(2);
    }

    @Test
    public void popRestituisceUltimo() { verifica(stack.pop() == 2); }

    @Test(atteso = NoSuchElementException.class)
    public void popSuVuoto() { stack.clear(); stack.pop(); }

    @Test
    public void questoFallisce() { verifica(stack.peek() == 1); }

    public void nonEUnTest() { throw new IllegalStateException("mai chiamato"); }

    static void verifica(boolean condizione) {
        if (!condizione) throw new AssertionError("condizione falsa");
    }
}
```

```
popRestituisceUltimo     ok
  popSuVuoto               ok (NoSuchElementException)
  questoFallisce           FALLITO: java.lang.AssertionError: condizione falsa
ProvaStack: 2/3 passati
```

JUnit vero fa molte cose in più (scopre le classi di test in un intero progetto, ha un vocabolario di asserzioni, misura i tempi, produce rapporti per l'IDE, gestisce i test parametrici), ma il nucleo è questo: annotazioni che marcano, reflection che trova e chiama. E un dettaglio del nostro runner, l'istanza nuova per ogni test, è una scelta di progetto di JUnit che spiega perché i test non possono contare su campi scritti da un test precedente.

## 4.5 Un oggetto senza classe: `Proxy`

Finora la reflection ci ha permesso di chiamare metodi che non conoscevamo. Il passo successivo è il contrario: *ricevere* chiamate a metodi che non abbiamo scritto. Supponiamo di voler registrare ogni chiamata fatta a una `List`, senza scrivere una classe che implementa i venticinque metodi di `List` inoltrando ciascuno. `java.lang.reflect.Proxy` costruisce a tempo di esecuzione una classe che implementa le interfacce che vogliamo e che, per ogni metodo, chiama un unico oggetto nostro, l'`InvocationHandler`, passandogli il `Method` chiamato e gli argomenti:

```java
@SuppressWarnings("unchecked")
static <T> T conLog(T vero, Class<T> interfaccia) {
    InvocationHandler gestore = (proxy, metodo, argomenti) -> {
        String args = argomenti == null ? "()" : Arrays.toString(argomenti);
        System.out.println("-> " + metodo.getName() + args);
        Object risultato = metodo.invoke(vero, argomenti);
        System.out.println("<- " + risultato);
        return risultato;
    };
    ClassLoader loader = interfaccia.getClassLoader();
    Class<?>[] interfacce = { interfaccia };
    return (T) Proxy.newProxyInstance(loader, interfacce, gestore);
}

List<String> lista = conLog(new ArrayList<>(), List.class);
lista.add("a");
lista.add("b");
System.out.println(lista.size() + " " + lista);
Class<?> k = lista.getClass();
System.out.println(k.getName() + " " + Proxy.isProxyClass(k));
```

```
-> add[a]
<- true
-> add[b]
<- true
-> size()
<- 2
-> toString()
<- [a, b]
2 [a, b]
jdk.proxy1.$Proxy0 true
```

`newProxyInstance` vuole un class loader (quello dell'interfaccia va bene, e nel capitolo 5 vedremo perché conta), l'elenco delle interfacce da implementare e il gestore. L'oggetto restituito è di una classe che non esiste nel sorgente, `jdk.proxy1.$Proxy0`, generata al momento; è una `List` a tutti gli effetti, e ogni chiamata, compresa `toString` dentro la concatenazione, passa dal gestore, che qui la stampa e la inoltra all'oggetto vero. `argomenti` è `null`, non un array vuoto, per i metodi senza parametri: un dettaglio che morde chi scrive `argomenti.length`.




Il proxy implementa l'interfaccia; tutto ciò che riceve lo passa a un solo metodo nostro.

Il gestore può fare qualunque cosa prima e dopo l'inoltro, o non inoltrare affatto. Un secondo esempio unisce proxy e annotazioni: una cache per i metodi marcati `@Cache`, che memorizza il risultato per ogni combinazione di argomenti.

```
interface Calcoli {
    @Cache long fibonacci(int n);
    long somma(int a, int b);
}

static <T> T conCache(T vero, Class<T> interfaccia) {
    Map<List<Object>, Object> memoria = new HashMap<>();
    ClassLoader loader = interfaccia.getClassLoader();
    return (T) Proxy.newProxyInstance(loader, new Class<?>[] { interfaccia },
        (proxy, metodo, argomenti) -> {
            if (!metodo.isAnnotationPresent(Cache.class))
                return metodo.invoke(vero, argomenti);
            List<Object> chiave = new ArrayList<>();
            chiave.add(metodo.getName());
            chiave.addAll(Arrays.asList(argomenti));
            Object r = memoria.get(chiave);
            if (r == null) {
                r = metodo.invoke(vero, argomenti);
                memoria.put(chiave, r);
            }
            return r;
        });
}
```

```
giro 1: fibonacci(40) = 102334155 in 171 ms, somma = 5
giro 2: fibonacci(40) = 102334155 in 0 ms, somma = 5
```

Chi usa `Calcoli` non sa della cache; chi scrive `CalcoliLenti` non sa del proxy; l'annotazione sull'interfaccia è il punto in cui i due si accordano. È esattamente il modo in cui un framework aggiunge una transazione, un controllo di permessi o una misura di tempo attorno a un metodo marcato `@Transactional`, `@Secured` o `@Timed`: l'oggetto che l'applicazione riceve non è il suo, è un proxy che lo avvolge.

**Attenzione 4.2: Che cosa un proxy non può fare**

Un `Proxy` implementa *interfacce*: non può estendere una classe, quindi non si può incartare un `ArrayList` dichiarato come tale, solo come `List` (le librerie che incartano classi generano bytecode, capitolo 8). I metodi di `Object` passano anch'essi dal gestore: `toString`, `hashCode` e `equals`; e un `equals` inoltrato ingenuamente confronta l'oggetto vero con il proxy, e risulta falso perfino per `p.equals(p)`. Infine il gestore deve restituire un valore del tipo giusto: per un metodo che rende un primitivo, un `return null` diventa una `NullPointerException` nel chiamante, in un punto del codice che non ha nessun `null` in vista.

## 4.6 L'esempio guida: funzioni che si presentano

Il menù del Plot del capitolo 3 mostra i nomi dei metodi e delle classi, e disegna tutto sullo stesso intervallo. Con `@Plot`, le funzioni possono dire come vogliono apparire: sulle classi, dove `Target` ammette `TYPE`, e sui metodi statici di una classe di funzioni brevi:

```
@Plot(nome = "Gaussiana", xMin = -4, xMax = 4)
public class Gaussiana implements Funzione {
    public double f(double x) { return Math.exp(-x * x / 2); }
}

public class Polinomi {
    @Plot(nome = "x^2", xMin = -3, xMax = 3)
    public static double quadrato(double x) { return x * x; }

    @Plot(nome = "x sin(x)", xMin = -20, xMax = 20)
    public static double xsinx(double x) { return x * Math.sin(x); }

    public static double aiuto(double x) { return 0; }      // senza @Plot: fuori
}
```

Una voce del menù diventa un record con la funzione e la sua presentazione, e il catalogo del capitolo 3 legge l'annotazione dove c'è. Per i metodi vale una regola: se in una classe almeno un metodo porta `@Plot`, entrano nel menù solo quelli annotati (così `aiuto` resta fuori); se nessuno la porta, come in `Math`, entrano tutti con il nome del metodo e l'intervallo di default.

```
public record Voce(String nome, Funzione f, double xMin, double xMax) {
    @Override public String toString() { return nome; }
}

public void aggiungiMetodiDi(Class<?> classe) {
    List<Method> candidati = Arrays.stream(classe.getMethods())
          .filter(m -> Modifier.isStatic(m.getModifiers()))
          .filter(m -> m.getReturnType() == double.class)
          .filter(m -> m.getParameterCount() == 1)
          .filter(m -> m.getParameterTypes()[0] == double.class)
          .sorted(Comparator.comparing(Method::getName))
          .toList();
    boolean annotata = candidati.stream()
                                .anyMatch(m -> m.isAnnotationPresent(Plot.class));
    for (Method m : candidati) {
        Plot p = m.getAnnotation(Plot.class);
        if (annotata && p == null)
            continue;
        voci.add(p == null ? new Voce(m.getName() + "(x)", incarta(m), -10, 10)
                           : new Voce(p.nome(), incarta(m), p.xMin(), p.xMax()));
    }
}

public void aggiungiClasse(String nomeClasse) {
    try {
        Class<?> classe = Class.forName(nomeClasse);
        if (!Funzione.class.isAssignableFrom(classe))
            throw new IllegalArgumentException(nomeClasse + ": non Funzione");
        Funzione fn = (Funzione) classe.getDeclaredConstructor().newInstance();
        Plot p = classe.getAnnotation(Plot.class);
        voci.add(p == null ? new Voce(classe.getSimpleName(), fn, -10, 10)
                           : new Voce(p.nome(), fn, p.xMin(), p.xMax()));
    } catch (ReflectiveOperationException e) {
        throw new IllegalArgumentException("impossibile usare " + nomeClasse, e);
    }
}
```

La combo del Plot ora contiene oggetti `Voce` (il suo `toString` è ciò che si vede), e al cambio di selezione la finestra imposta l'intervallo e la funzione. Per la riga di stato usiamo un proxy: un `Contatore` che incarta la `Funzione` scelta e conta le chiamate a `f`, così da vedere quante valutazioni costa un ridisegno, senza toccare né il pannello né le funzioni.

```
public class Contatore {
    private final AtomicLong chiamate = new AtomicLong();

    public Funzione conta(Funzione vera) {
        return (Funzione) Proxy.newProxyInstance(Funzione.class.getClassLoader(),
            new Class<?>[] { Funzione.class },
            (proxy, metodo, argomenti) -> {
                if (metodo.getName().equals("f"))
                    chiamate.incrementAndGet();
                return metodo.invoke(vera, argomenti);
            });
    }

    public long chiamate() { return chiamate.get(); }
    public void azzera() { chiamate.set(0); }
}

private void disegna() {
    Voce voce = (Voce) cmbFunzione.getSelectedItem();
    Funzione contata = contatore.conta(voce.f());
    contatore.azzera();
    pnlPlot.setIntervallo(voce.xMin(), voce.xMax());
    pnlPlot.setFunzione(contata::f);
    pnlPlot.paintImmediately(0, 0, pnlPlot.getWidth(), pnlPlot.getHeight());
    lblStato.setText(voce + " su [" + voce.xMin() + ", " + voce.xMax() + "]: "
            + contatore.chiamate() + " valutazioni per il ridisegno");
}
```

Il menù mostra *Gaussiana*, *sin(x) / x*, *x sin(x)* accanto alle ventisei funzioni di `Math`, ciascuna sul suo intervallo, e la riga di stato dice "821 valutazioni per il ridisegno" (una per pixel di larghezza, più una per il punto sotto il mouse). Il contatore passa dal proxy e da `invoke` a ogni pixel: è il costo misurato nel capitolo 3, qualche microsecondo in tutto. Con questo il Plot ha finito di guardare dentro sé stesso: i due capitoli che seguono lo aprono al codice che arriva da fuori, prima quello già compilato (capitolo 5), poi quello scritto dall'utente (capitolo 6), e in entrambi i casi `@Plot` sarà il modo in cui una funzione nuova dice come chiamarsi.

**Ricetta: un'annotazione che il programma legge**

1. dichiararla con `@interface`, con `@Retention(RUNTIME)` per prima cosa e `@Target` con i soli posti in cui ha senso;
2. elementi con i tipi ammessi, default per quelli opzionali; mai `null`: un valore sentinella al suo posto;
3. chi la legge: un ciclo sui membri con `isAnnotationPresent` o `getAnnotation`, e una decisione per ogni elemento trovato;
4. la ricerca una volta, all'avvio (regola del capitolo 3): il risultato in una lista o in una mappa, non un `getAnnotation` a ogni uso.

## Esercizi

### Esercizio 4.1: Retention (⋆)

Per ciascuna annotazione dire quale retention le serve e perché: (a) una `@Test` letta da un runner; (b) una `@Override`; (c) una `@Generated` che uno strumento legge nei `.class` per non ricompilare ciò che ha generato; (d) una `@Plot` letta dal catalogo. Che cosa restituisce `getAnnotation` per un'annotazione con retention `CLASS`?

### Esercizio 4.2: Elementi ammessi (⋆)

Quali di queste dichiarazioni il compilatore rifiuta, e perché? (a) `String nome() default null;` (b) `Class<?> tipo();` (c) `List<String> tag();` (d) `String[] tag() default {};` (e) `Object valore();` (f) `Plot interno();`

### Esercizio 4.3: Chi la vede (⋆⋆)

Un'annotazione `@Marca`, con retention `RUNTIME` e senza `@Inherited`, è scritta su una classe `Base` e su un suo metodo `m()`. `Derivata extends Base` ridefinisce `m()` senza annotarlo. Dire il valore di: (a) `Derivata.class.isAnnotationPresent(Marca.class)`; (b) lo stesso con `@Inherited` sulla dichiarazione di `Marca`; (c) `Derivata.class.getMethod("m").isAnnotationPresent(Marca.class)`, con e senza `@Inherited`; (d) `Base.class.getMethod("m").isAnnotationPresent(Marca.class)`.

### Esercizio 4.4: Il runner che non trova nulla (⋆⋆)

Il mini-JUnit stampa "0/0 passati" su una classe che ha quattro metodi con `@Test`. Elencare almeno tre cause possibili, dalla più probabile, e per ciascuna dire come si verifica. Poi: un test è dichiarato `private`: che cosa succede nel nostro runner, e come si cambia il runner perché lo esegua lo stesso?

### Esercizio 4.5: Il proxy e i primitivi (⋆⋆)

Un gestore scritto per "spegnere" un oggetto restituisce sempre `null` senza inoltrare. Il proxy implementa un'interfaccia con `String nome()`, `int conta()` e `void azzera()`. Dire che cosa succede chiamando ciascuno dei tre metodi sul proxy, e come si scrive un gestore "neutro" che funziona per tutti e tre.

*I problemi che seguono chiedono un programma completo, in ordine crescente di difficoltà. Le soluzioni sono nel repository delle soluzioni del corso, nel progetto e nel package indicati.*

### Esercizio 4.6: Gli autori (⋆)

Dichiarare un'annotazione `@Autore(nome, anno)` ripetibile, applicabile a classi e metodi, e usarla su tre o quattro classi di prova (alcune con due autori, alcune senza). Scrivere un programma che riceve un elenco di nomi di classi, legge le annotazioni e stampa un indice per autore: per ciascun autore, in ordine alfabetico, le classi e i metodi a cui ha lavorato con l'anno.

Soluzione: progetto `sol-dxtr-01-c04`, package `it.unibs.pajc.redolfi.e06`.

### Attenzione 4.3: Ripetibile vuol dire due annotazioni

Con `@Repeatable`, due `@Autore` sullo stesso elemento vengono impacchettate dal compilatore in una sola annotazione contenitore, e `getAnnotation(Autore.class)` restituisce `null`, perché sull'elemento "c'è" solo il contenitore. Si usa `getAnnotationsByType`, che dà l'array in entrambi i casi.

### Esercizio 4.7: Il mini-JUnit completo (⋆⋆)

Estendere il runner della sezione 4.4: `@Dopo` eseguito dopo ogni test anche se fallisce; `@Test(timeout = ms)` che dichiara fallito un test più lento del limite (il test gira in un thread a parte, e il runner aspetta con `join(ms)`); il nome della classe da riga di comando; un riepilogo con il tempo di ciascun test e il totale, e lo stato di uscita del processo diverso da zero se qualcosa fallisce. Scrivere una classe di test su una `String` o su una collezione che eserciti tutte le annotazioni.

Soluzione: progetto `sol-dxtr-01-c04`, package `it.unibs.pajc.redolfi.e07`.

### Esercizio 4.8: Il validatore (⋆⋆)

Dichiarare `@NonNullo`, `@Intervallo(min, max)` per i campi numerici e `@Lunghezza(min, max)` per le stringhe, e scrivere `static List<String> valida(Object o)` che restituisce la lista delle violazioni, una riga ciascuna con nome del campo, valore e regola violata (lista vuota se tutto è in ordine). I campi possono essere privati. Provarlo con un record o una classe `Ordine` con alcuni campi annotati e un paio di oggetti sbagliati in modi diversi.

Soluzione: progetto `sol-dxtr-01-c04`, package `it.unibs.pajc.redolfi.e08`.

**Esercizio 4.9: Misurato (⋆⋆⋆)**

Scrivere `static <T> T Misurato.di(T oggetto, Class<T> interfaccia)` che restituisce un proxy che accumula, per ogni metodo dell'interfaccia, il numero di chiamate e il tempo totale, e un metodo `Misurato.rapporto(Object proxy)` che stampa la tabella (metodo, chiamate, tempo medio) per quel proxy. Il proxy deve funzionare correttamente anche con `toString`, `equals` e `hashCode`, e deve far arrivare al chiamante l'eccezione *originale* di un metodo che fallisce, non una `InvocationTargetException`. Provarlo su una `Map`.
*Suggerimento: il rapporto ha bisogno di risalire dal proxy al suo gestore:* `Proxy.getInvocationHandler(proxy)`.

Soluzione: progetto `sol-dxtr-01-c04`, package `it.unibs.pajc.redolfi.e09`.

## Risposte

**Risposta 4.1.** (a) `RUNTIME`: il runner la legge con la reflection. (b) `SOURCE`: serve solo al compilatore. (c) `CLASS`: deve stare nel file compilato, ma nessuno la legge a runtime, ed è inutile caricarla in memoria. (d) `RUNTIME`, come (a). Con `CLASS`, `getAnnotation` restituisce `null`, senza eccezioni.

**Risposta 4.2.** (a) rifiutata: `null` non è ammesso come default. (b) ammessa. (c) rifiutata: `List` non è fra i tipi ammessi; si usa un array, come in (d), ammessa (array vuoto come default). (e) rifiutata: `Object` non è ammesso. (f) ammessa: un'annotazione può avere elementi di tipo annotazione.

**Risposta 4.3.** (a) falso: senza `@Inherited` le annotazioni di classe non passano alle sottoclassi. (b) vero. (c) falso in entrambi i casi: `@Inherited` riguarda solo le classi, e il metodo ridefinito è un altro metodo, senza annotazione. (d) vero, è il metodo che la porta.

**Risposta 4.4.** Cause: manca `@Retention(RUNTIME)` sulla dichiarazione di `@Test` (si verifica stampando le annotazioni di un metodo con `getAnnotations()`: nessuna); si è importato un altro `Test` (per esempio quello di JUnit) e il runner cerca il nostro (si confronta il nome dell'annotazione trovata con quello della nostra classe `Test`); i test sono in una superclasse e il runner usa `getDeclaredMethods` (si guarda `getMethods`). Un test `private` viene trovato da `getDeclaredMethods` ma `invoke` lancia `IllegalAccessException`, che nel nostro runner non è catturata e lo ferma; per eseguirlo, `m.setAccessible(true)` prima di `invoke`.

**Risposta 4.5.** `nome()` restituisce `null`, legittimo per una `String`; `azzera()` va bene, `void` ignora il valore; `conta()` fa `NullPointerException` nel chiamante, perché il proxy deve convertire il `null` in un `int`. Un gestore neutro guarda `metodo.getReturnType()`: se è un primitivo restituisce il suo zero (per esempio `Array.get(Array.newInstance(tipo, 1), 0)`, che dà il default incartato di qualunque primitivo), altrimenti `null`.

Capitolo 5

# Da dove vengono le classi: class loader, jar e plugin

Fin qui il Plot ha guardato dentro classi che aveva già: `Class.forName` del capitolo 2 cerca nel class path del programma, e una classe scritta da un collega non c'è. Questo capitolo risponde alla seconda domanda del capitolo 1: come si carica codice compilato che non c'era quando il programma è stato compilato. L'oggetto che lo fa è il *class loader*, il passaggio centrale dell'immagine guida (sorgente, `.class`, loader, `Class`), che finora abbiamo attraversato senza guardarlo. Vedremo come i loader sono organizzati, come se ne crea uno che guarda in una cartella o in un `.jar`, una regola sorprendente (la stessa classe caricata da due loader è due tipi diversi) e il meccanismo standard con cui Java trova le implementazioni di un'interfaccia dentro i `.jar`. Alla fine il Plot avrà la sua cartella `plugin/` e un pulsante *Ricarica*: la terza richiesta dell'utente.

## 5.1 Chi legge i `.class`

Ogni oggetto `Class` è stato costruito da un class loader, e lo ricorda: `getClassLoader()` lo restituisce. Un programma che lo chiede per alcune classi scopre che i loader sono più di uno:

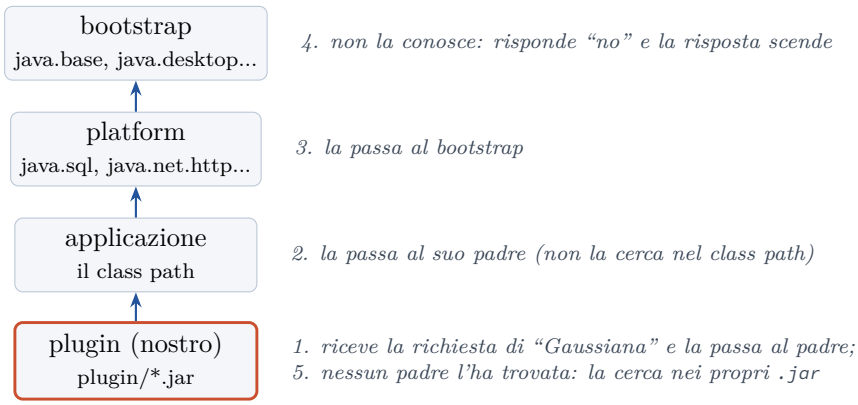
```
static void stampa(String che, Class<?> c) {
    ClassLoader l = c.getClassLoader();
    String nome = l == null ? "bootstrap (null)" : l.getName();
    System.out.printf("%-20s %s%n", che, nome);
}

stampa("String", String.class);
stampa("java.sql.Connection", java.sql.Connection.class);
stampa("ChiCarica", ChiCarica.class);
stampa("int[]", int[].class);
Supplier<String> s = () -> "ciao";
stampa("lambda", s.getClass());
for (ClassLoader l = ChiCarica.class.getClassLoader(); l != null; l =
    l.getParent())
    String tipo = l.getClass().getSimpleName();
    System.out.println("  " + l.getName() + " (" + tipo + ")");
```

```
String                bootstrap (null)
java.sql.Connection   platform
ChiCarica             app
int[]                 bootstrap (null)
lambda                app
  app  (AppClassLoader)
  platform  (PlatformClassLoader)
```

Tre loader, in catena. Il *bootstrap* è dentro la JVM, scritto in C, e carica il cuore della libreria standard (`java.base`: `String`, `Object`, le collezioni); non ha un oggetto Java che lo rappresenti, e `getClassLoader()` restituisce `null` per le sue classi. Il *platform* loader carica i moduli del JDK che non sono nel nucleo (`java.sql`, `java.net.http`, `java.scripting`, ...; Swing invece sta in `java.desktop`, che è ancora del bootstrap). Il loader dell'*applicazione* ("app", o *system class loader*) carica le classi del class path, cioè le nostre e quelle delle librerie: anche le classi delle lambda, generate a runtime, appartengono al loader della classe che le contiene. Ogni loader ha un *padre* (`getParent()`), e la catena finisce nel bootstrap.

La catena non è un dettaglio: è il modo in cui i loader decidono chi carica che cosa. Quando a un loader viene chiesta una classe, prima la chiede al padre, e solo se il padre non la trova la cerca lui, nei posti che conosce. È la *delega parent-first*, e il motivo è la sicurezza e la coerenza: una classe nostra chiamata `java.lang.String` nel class path non sostituirebbe mai quella vera, perché la richiesta sale fino al bootstrap, che la trova per primo; e in tutta la JVM c'è una sola `String`, qualunque loader l'abbia chiesta.




La delega: ogni richiesta sale fino in cima, e scende solo se nessun padre risponde.

**Definizione 5.1: Class loader**

Un class loader è un oggetto, sottoclasse di `java.lang.ClassLoader`, che sa trovare i byte di una classe a partire dal suo nome binario e consegnarli alla JVM, che ne fa l'oggetto `Class`. Ha un padre a cui delega prima di cercare, e ricorda le classi che ha caricato: una classe, per un loader, si carica una volta sola, e le richieste successive restituiscono lo stesso oggetto.

## 5.2 Un loader che guarda altrove

Per caricare classi da un posto che non è nel class path serve un loader che conosca quel posto. La libreria ne fornisce uno pronto, `URLClassLoader`: riceve un elenco di URL (cartelle, che devono finire con `/`, o file `.jar`) e il padre, e cerca le classi lì. Un `.class` di `Gaussiana` compilato a parte, in una cartella `out-plugin/` con la struttura dei package, si carica così:

```
URL cartella = Path.of("out-plugin").toUri().toURL();
ClassLoader padre = DueLoader.class.getClassLoader();
URLClassLoader a = new URLClassLoader("A", new URL[] { cartella }, padre);
Class<?> ca = Class.forName("it.unibs.pajc.plot.funzioni.Gaussiana", true, a);
Class<?> ca2 = a.loadClass("it.unibs.pajc.plot.funzioni.Gaussiana");
System.out.println("stesso loader, stessa classe: " + (ca == ca2));     // true
```

Due modi di chiedere la classe. `forName` a tre argomenti, con il loader e `initialize` a `true`, carica e inizializza, ed è la forma del capitolo 2 con il loader scelto da noi; `loadClass` del loader carica senza inizializzare (l'inizializzazione arriverà al primo uso), e serve a ispezionare senza eseguire. Nessuno dei due ricarica: il loader `A` ha costruito `Gaussiana` una volta, e `ca == ca2`. Se il file nella cartella cambia, `A` non se ne accorge mai, e questo è il punto della sezione seguente.

Il padre va scelto con cura. Il nostro `Gaussiana` implementa `Funzione`, che sta nel programma principale: quando il loader `A` la carica, la JVM deve risolvere `Funzione`, e `A` la chiede al padre, che è il loader dell'applicazione e ce l'ha. Con `null` come padre (il bootstrap) la classe `Gaussiana` si caricherebbe, ma al momento di collegarla `NoClassDefFoundError: it/unibs/pajc/plot/Funzione`, perché il bootstrap non conosce le nostre interfacce.

## 5.3 Un nome, due loader, due tipi

Creiamo un secondo loader, `B`, identico ad `A`, e chiediamo a entrambi la stessa classe dalla stessa cartella:

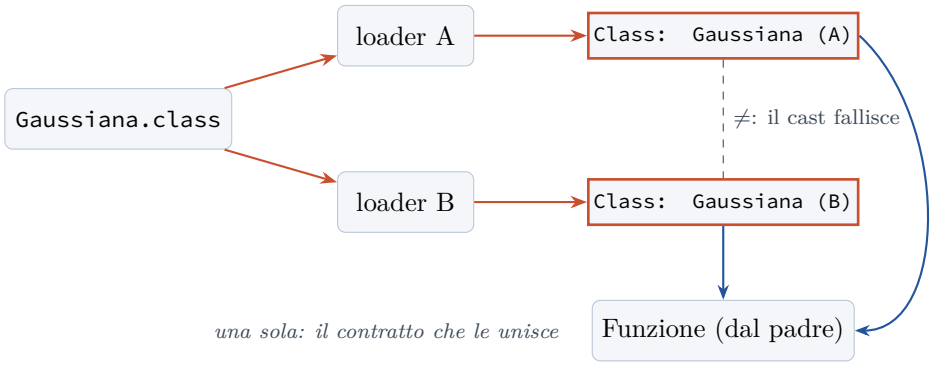
```
URLClassLoader b = new URLClassLoader("B", new URL[] { cartella }, padre);
Class<?> cb = Class.forName(nome, true, b);
System.out.println("loader diversi, stessa classe? " + (ca == cb));
Object oa = ca.getDeclaredConstructor().newInstance();
try {
    cb.cast(oa);
} catch (ClassCastException e) {
    System.out.println("cast: " + e.getMessage());
}
System.out.println("Funzione di A == Funzione del padre: "
        + (ca.getInterfaces()[0] ==
    Class.forName("it.unibs.pajc.plot.Funzione")));
```

```
loader diversi, stessa classe? false
cast: Cannot cast it.unibs.pajc.plot.funzioni.Gaussiana
  to it.unibs.pajc.plot.funzioni.Gaussiana
Funzione di A == Funzione del padre: true
```

Stessi byte, stesso nome, due oggetti `Class` diversi, e un cast che fallisce con un messaggio che sembra assurdo: "non posso convertire `Gaussiana` in `Gaussiana`". È la regola fondamentale dei loader.

**Definizione 5.2: L'identità di una classe**

Nella JVM una classe è identificata dalla coppia (nome binario, loader che l'ha definita). Due loader che caricano lo stesso `.class` producono due classi distinte, non assegnabili l'una all'altra, con campi statici separati e istanze incompatibili. L'unicità dell'oggetto `Class` promessa nel capitolo 2 vale *per loader*.




Stessi byte, due loader, due tipi; l'interfaccia condivisa deve venire dal padre.

La regola ha due conseguenze pratiche, una buona e una da ricordare. Quella buona: *ricaricare* una classe è possibile, basta un loader nuovo. Un loader non può dimenticare una classe, ma un secondo loader la legge da capo dal disco, e se nel frattempo il `.jar` è cambiato si ottiene la versione nuova; è così che un server applicativo ridistribuisce un'applicazione senza riavviare, ed è così che il pulsante *Ricarica* del Plot funzionerà. Quella da ricordare: il programma principale e le classi caricate dal loader nuovo si parlano solo attraverso tipi che *entrambi* vedono dallo stesso loader, e cioè dal padre. `Funzione` sta nel programma, i loader `A` e `B` la delegano al padre, e per questo `ca.getInterfaces()[0]` è *la* `Funzione`: il Plot può fare il cast a `Funzione` di un oggetto di `Gaussiana (A)` o di `Gaussiana (B)`, e chiamare `f`. Se invece il `.jar` del plugin contenesse una propria copia di `Funzione.class`, la delega la troverebbe comunque prima nel padre (parent-first), e andrebbe bene; ma se il programma caricasse `Funzione` *anch'esso* da un loader figlio, ci sarebbero due `Funzione`, e il cast fallirebbe con lo stesso messaggio assurdo.

**Attenzione 5.1: "X cannot be cast to X"**

Quando un `ClassCastException` nomina lo stesso tipo da entrambe le parti, la causa è sempre questa: due loader. Succede quando un'interfaccia o una classe condivisa è stata messa anche nel `.jar` di un plugin *e* il loader del plugin non delega al loader del programma (padre sbagliato, o un loader scritto a mano che cerca prima da sé), oppure quando si confrontano oggetti di due ricariche successive. La cura è nel progetto: il contratto in un solo posto, nel programma, e i loader figli con il padre giusto.

E lo scarico? Una classe vive finché qualcuno può raggiungerla: il suo loader, le sue istanze, altre classi che la riferiscono. Quando loader, classi e istanze diventano tutti irraggiungibili, il garbage collector li elimina insieme, e la memoria della classe torna libera. Si può verificare con un riferimento debole, che non tiene in vita ciò che indica:

```
WeakReference<Class<?>> ref = new WeakReference<>(cb);
b.close();                                  // chiude i file .jar aperti dal loader
b = null;                                   // ne' il loader...
cb = null;                                  // ...ne' la classe: nessuno li
    riferisce
for (int i = 0; i < 10 && ref.get() != null; i++) {
    System.gc();
    Thread.sleep(50);
}
System.out.println("classe di B scaricata? " + (ref.get() == null));   // true
```

`close()` di `URLClassLoader` chiude i `.jar` che il loader teneva aperti (su Windows, finché sono aperti non si possono sovrascrivere); lo scarico vero lo fa il garbage collector, quando può. Un programma che ricarica plugin molte volte e tiene da qualche parte, anche solo in una lista di menù, un oggetto della versione vecchia, tiene in vita anche il suo loader e tutte le sue classi: la perdita di memoria più classica dei server con il *hot reload*.

## 5.4 Il `.jar`

Un `.jar` è un file zip con una convenzione: dentro, le cartelle dei package con i `.class` (`it/unibs/pajc/plot/funzioni/Gaussiana.class`), eventuali risorse, e una cartella `META-INF` con il *manifesto*, `MANIFEST.MF`, un file di testo con coppie chiave-valore. Lo crea lo strumento `jar` del JDK, a partire da una cartella che ha già la struttura giusta (quella in cui il compilatore ha scritto i `.class`):

```
$ jar --create --file plugin/gauss.jar -C out-plugin .
$ jar --list --file plugin/gauss.jar
META-INF/
META-INF/MANIFEST.MF
META-INF/services/
META-INF/services/it.unibs.pajc.plot.Funzione
it/
it/unibs/
it/unibs/pajc/
it/unibs/pajc/plot/
it/unibs/pajc/plot/funzioni/
it/unibs/pajc/plot/funzioni/Gaussiana.class
it/unibs/pajc/plot/funzioni/Logistica.class
```

`-C out-plugin .` dice "entra in `out-plugin` e prendi tutto": senza, il `.jar` conterrebbe `out-plugin/it/...` e nessun loader troverebbe le classi, perché il percorso dentro l'archivio deve coincidere con il package. Eclipse fa la stessa cosa con *File → Export → JAR file*. Il manifesto generato contiene solo la versione; due chiavi meritano di essere conosciute. `Main-Class: it.unibs.pajc.plot.PlotApp` rende il `.jar` eseguibile con `java -jar plot.jar`; `Class-Path: lib/a.jar lib/b.jar` elenca altri `.jar` da cui dipende, relativi alla sua posizione, che il loader dell'applicazione aggiunge da solo. Per leggere un `.jar` da programma c'è `java.util.jar.JarFile`, con `entries()` per le voci e `getManifest()`; per scriverne uno, `JarOutputStream`, che useremo nel capitolo 6 per salvare la funzione dell'utente.

## 5.5 Plugin alla maniera di Java: `ServiceLoader`

Con `URLClassLoader` sappiamo caricare un `.jar`; resta da sapere *quali* classi, là dentro, sono funzioni per il Plot. Si potrebbe chiedere al collega di scrivere i nomi in un file di testo, o di chiamare la classe in un modo convenuto. Java ha già la convenzione, usata dal JDK stesso per i driver JDBC, i provider di cifratura, i motori di script: un'interfaccia è un *servizio*, le classi che la implementano sono i *provider*, e ogni `.jar` dichiara i propri provider in un file dentro `META-INF/services/` che si chiama *esattamente* come il nome binario dell'interfaccia e contiene i nomi binari delle classi, uno per riga:

```
$ cat META-INF/services/it.unibs.pajc.plot.Funzione
it.unibs.pajc.plot.funzioni.Gaussiana
it.unibs.pajc.plot.funzioni.Logistica
```

Dal lato del programma, `ServiceLoader.load(Funzione.class, loader)` restituisce un oggetto iterabile che cerca, nel loader indicato e nei suoi padri, tutti i file con quel nome, legge i nomi delle classi, le carica, verifica che implementino l'interfaccia, le istanzia con il costruttore pubblico senza argomenti e le consegna una alla volta:

```
for (Funzione f : ServiceLoader.load(Funzione.class, loader))
    System.out.println(f.getClass().getName() + ": f(0) = " + f.f(0));
```

Il caricamento è pigro: le classi si caricano e si istanziano man mano che l'iterazione avanza, e un provider rotto (classe mancante, costruttore assente) lancia `ServiceConfigurationError` al suo turno, non prima. È tutto ciò che serve a un sistema di plugin: il programma conosce l'interfaccia, il `.jar` dichiara le implementazioni, nessuno dei due conosce i nomi dell'altro.

**Attenzione 5.2: Il file che non si chiama come l'interfaccia**

`ServiceLoader` non segnala nulla se non trova file: restituisce zero provider, come se il `.jar` fosse vuoto. Un file chiamato `Funzione` invece di `it.unibs.pajc.plot.Funzione`, o messo in `META-INF/service` senza la `s`, o con il nome di una classe sbagliato di una lettera, dà un menù senza plugin e nessun indizio. Quando "il plugin non si carica", la prima verifica è `jar -list` e il nome del file, lettera per lettera; la seconda è che la classe sia `public` con un costruttore `public` senza argomenti, altrimenti l'errore arriva, ma solo quando si itera.

## 5.6 L'esempio guida: la cartella `plugin/`

Mettiamo insieme i pezzi. Il collega scrive le sue funzioni in un progetto a parte, che ha nel class path il `.jar` (o la cartella `bin`) del Plot per vedere `Funzione` e `@Plot`:

```
package it.unibs.pajc.plot.funzioni;

@Plot(nome = "logistica", xMin = -6, xMax = 6)
public class Logistica implements Funzione {
    public double f(double x) {
        return 1 / (1 + Math.exp(-x));
    }
}
```

Aggiunge il file `META-INF/services/it.unibs.pajc.plot.Funzione` con i nomi delle sue classi, crea il `.jar` e lo copia nella cartella `plugin/` accanto al Plot. Dal lato del Plot, una classe `Plugin` fa tutto il lavoro di questo capitolo:

```
public class Plugin {

    private final Path cartella;
    private URLClassLoader loader;                  // l'ultima ricarica, da
    chiudere

    public Plugin(Path cartella) {
        this.cartella = cartella;
    }

    /** Legge tutti i .jar della cartella e restituisce le loro Funzione. */
    public List<Voce> ricarica() throws IOException {
        if (loader != null)
            loader.close();
        URL[] jar;
        try (Stream<Path> file = Files.list(cartella)) {
            jar = file.filter(p -> p.toString().endsWith(".jar"))
                      .map(Plugin::url).toArray(URL[]::new);
        }
        loader = new URLClassLoader("plugin", jar,
    Funzione.class.getClassLoader());
        List<Voce> voci = new ArrayList<>();
        for (Funzione f : ServiceLoader.load(Funzione.class, loader)) {
            Plot p = f.getClass().getAnnotation(Plot.class);
            voci.add(p == null ? new Voce(f.getClass().getSimpleName(), f, -10,
    10)
                               : new Voce(p.nome(), f, p.xMin(), p.xMax()));
        }
        return voci;
    }

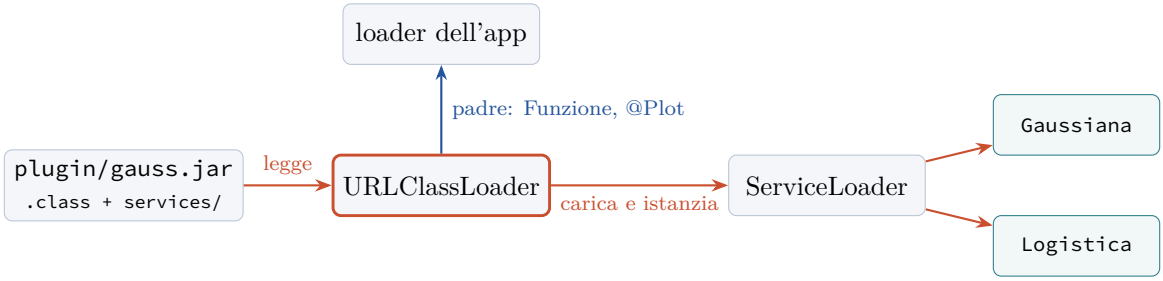
    private static URL url(Path p) {
        try {
            return p.toUri().toURL();
        } catch (MalformedURLException e) {
            throw new IllegalArgumentException(p.toString(), e);
        }
    }
}
```

Ogni `ricarica()` chiude il loader precedente e ne crea uno nuovo su tutti i `.jar` che trova in quel momento, con il loader di `Funzione` come padre (sezione 5.3): è ciò che permette di sostituire un `.jar` e premere *Ricarica* senza riavviare. `ServiceLoader` fa la scoperta, e `@Plot` del capitolo 4 fornisce nome e intervallo, con lo stesso codice usato per le classi interne: per il catalogo un plugin non è diverso da una funzione scritta nel programma. La finestra aggiunge alle voci del catalogo quelle dei plugin, e un pulsante:

```
private final Plugin plugin = new Plugin(Path.of("plugin"));

private void ricarica() {
    try {
        List<Voce> voci = new ArrayList<>(catalogo.voci());
        voci.addAll(plugin.ricarica());
        Voce[] modello = voci.toArray(new Voce[0]);
        cmbFunzione.setModel(new DefaultComboBoxModel<>(modello));
        int daPlugin = voci.size() - catalogo.voci().size();
        lblStato.setText(voci.size() + " funzioni, " + daPlugin + " da plugin/");
    } catch (IOException e) {
        lblStato.setText("impossibile leggere plugin/: " + e.getMessage());
    }
}
```

Con `gauss.jar` nella cartella il menù mostra *Gaussiana (plugin)* e *logistica* dopo le funzioni interne; se il collega manda una versione nuova del `.jar`, la si copia sopra la vecchia e *Ricarica* la prende. Le voci della ricarica precedente vengono sostituite nel modello della combo, così nessun oggetto della versione vecchia resta raggiungibile e il suo loader può essere scaricato (sezione 5.3).




Dal `.jar` alle voci del menù: il loader carica, `ServiceLoader` scopre, `@Plot` presenta.

Il Plot risponde ora alla terza richiesta dell'utente: una cartella in cui chiunque può mettere funzioni scritte in Java. Resta la prima, e la più ambiziosa: la funzione scritta nel campo di testo. Il capitolo 6 la risolve con il compilatore, e il loader di questo capitolo è l'ultimo anello della sua catena.

**Ricetta: una cartella di plugin**

1. il *contratto* nel programma: un'interfaccia (e le annotazioni che la accompagnano) che i plugin implementano; mai una copia dentro i `.jar`;
2. il plugin: un progetto a parte con il programma nel class path, le classi pubbliche con costruttore pubblico senza argomenti, il file `META-INF/services/<nome.binario.dell.interfaccia>` con i nomi delle classi;
3. il `.jar` da una cartella con la struttura dei package (`jar -create -C bin .`), copiato nella cartella dei plugin;
4. nel programma, a ogni ricarica: chiudere il loader vecchio, un `URLClassLoader` nuovo su tutti i `.jar` con il loader del contratto come padre, `ServiceLoader.load(Contratto.class, loader)`;
5. non tenere riferimenti agli oggetti della ricarica precedente, se si vuole che la memoria torni libera.

## Esercizi

**Esercizio 5.1: Chi carica (⋆)**

Senza eseguire, dire quale loader (bootstrap, platform, applicazione) ha caricato: (a) `java.util.ArrayList`; (b) `javax.swing.JFrame`; (c) una classe nostra `PlotApp`; (d) la classe di una lambda scritta dentro `PlotApp`; (e) `String[]`; (f) una classe della libreria Gson messa nel class path.

**Esercizio 5.2: La delega (⋆)**

Un `URLClassLoader L`, con il loader dell'applicazione come padre, punta a una cartella che contiene `java/lang/String.class` (una versione "nostra") e `x/Mia.class`. Dire che cosa restituisce `L.loadClass("java.lang.String")`, e quale loader risulta da `getClassLoader()` della classe ottenuta; e lo stesso per `L.loadClass("x.Mia")`.

**Esercizio 5.3: Quante classi (⋆⋆)**

Un programma esegue `ricarica()` del Plot tre volte, con lo stesso `gauss.jar`, e ogni volta mette le voci in una lista che non svuota mai. Quanti oggetti `Class` di nome `Gaussiana` esistono alla fine? Quanti loader `plugin`? Che cosa succede confrontando con `==` le classi di due voci di ricariche diverse, e con `instanceof Funzione`?

**Esercizio 5.4: Il cast impossibile (⋆⋆)**

Il collega, per far compilare il suo plugin senza il Plot nel class path, ha copiato `Funzione.java` nel proprio progetto, nello stesso package. Il `.jar` contiene quindi `it/unibs/pajc/plot/Funzione.class`. Dire se il Plot carica le sue funzioni, e perché; poi dire che cosa succederebbe se `Plugin` creasse il loader con `null` come padre.

**Esercizio 5.5: Zero provider (⋆⋆)**

`ServiceLoader.load(Funzione.class, loader)` non restituisce nulla, senza errori. Elencare quattro cause possibili e, per ciascuna, il comando o la riga di codice con cui la si verifica.

*I problemi che seguono chiedono un programma completo, in ordine crescente di difficoltà. Le soluzioni sono nel repository delle soluzioni del corso, nel progetto e nel package indicati.*

**Esercizio 5.6: Chi ha caricato chi (⋆)**

Scrivere un programma che riceve nomi di classi dalla riga di comando e per ciascuna stampa il loader che l'ha caricata, la catena dei suoi padri fino al bootstrap, e, se è una classe nostra, il percorso del file `.class` da cui viene (`getProtectionDomain().getCodeSource().getLocation()`). Provarlo con classi del JDK, con una classe del programma e con una lambda.

Soluzione: progetto `sol-dxtr-01-c05`, package `it.unibs.pajc.redolfi.e06`.

**Attenzione 5.3: Il loader è `null`**

Per le classi del bootstrap `getClassLoader()` restituisce `null`, e anche `getCodeSource()` può esserlo: un programma che stampa la catena con `l.getParent().getName()` senza controlli si ferma sulla prima classe di `java.lang`.

**Esercizio 5.7: Un lanciatore (⋆⋆)**

Scrivere un programma `Lancia` che riceve una cartella (o un `.jar`) e il nome di una classe, crea un `URLClassLoader` su quella cartella, carica la classe, cerca il suo `public static void main(String[])` e lo chiama per reflection con gli argomenti rimanenti; se la classe non c'è o non ha un `main`, un messaggio chiaro. Provarlo lanciando una classe compilata in un altro progetto, e una classe che nel `main` lancia un'eccezione (che cosa si vede, e come la si mostra "pulita").

Soluzione: progetto `sol-dxtr-01-c05`, package `it.unibs.pajc.redolfi.e07`.

**Esercizio 5.8: Un'applicazione a comandi (⋆⋆)**

Un'interfaccia `Comando` con `String nome()`, `String aiuto()` e `void esegui(List<String> argomenti)`. Scrivere un programma a riga di comando che, all'avvio, carica da una cartella `plugin/` tutti i `Comando` trovati con `ServiceLoader`, costruisce il menù (`aiuto` elenca i comandi disponibili con la loro descrizione, `ricarica` rilegge la cartella) ed esegue i comandi digitati. Scrivere in un progetto a parte un `.jar` con due o tre comandi (`data`, `conta` le parole di un argomento, `inverti`).

Soluzione: progetto `sol-dxtr-01-c05`, package `it.unibs.pajc.redolfi.e08`.

**Esercizio 5.9: Ricarica a caldo (⋆⋆⋆)**

Scrivere due versioni di una classe `Saluto implements Supplier<String>`, che restituiscono `"versione 1"` e `"versione 2"`, compilate in due cartelle diverse. Un programma le carica con due loader, mostra che sono due classi con lo stesso nome, che un'istanza dell'una non è assegnabile all'altra, e che entrambe sono `Supplier<String>` del programma; poi abbandona il primo loader e verifica con una `WeakReference` che la sua classe viene scaricata, e mostra un caso in cui *non* viene scaricata (una sua istanza tenuta in una lista).
*Suggerimento: per "abbandonare" davvero servono tre `null`: il loader, la classe e l'istanza; e qualche `System.gc()` in un ciclo.*

Soluzione: progetto `sol-dxtr-01-c05`, package `it.unibs.pajc.redolfi.e09`.

## Risposte

**Risposta 5.1.** (a) bootstrap (`java.base`). (b) bootstrap: `java.desktop` è fra i moduli del nucleo, come si verifica con `JFrame.class.getClassLoader()`, che dà `null`. (c) applicazione. (d) applicazione: la classe della lambda appartiene al loader della classe che la contiene. (e) bootstrap, come il suo tipo di elemento. (f) applicazione: tutto il class path.

**Risposta 5.2.** `java.lang.String`: la richiesta sale fino al bootstrap, che la trova; la classe è quella vera e `getClassLoader()` è `null`. La copia nella cartella non viene mai letta. `x.Mia`: nessun padre la conosce, la carica `L`, e `getClassLoader()` è `L`.

**Risposta 5.3.** Tre classi `Gaussiana` e tre loader, tutti vivi perché le voci della lista tengono le istanze, che tengono le classi, che tengono i loader. Le classi di due ricariche confrontate con `==` sono diverse; `instanceof Funzione` è vero per tutte, perché `Funzione` viene dal padre ed è una sola.

**Risposta 5.4.** Le carica: il loader del plugin, prima di cercare `Funzione` nel `.jar`, la chiede al padre, che ce l'ha; la copia nel `.jar` viene ignorata e la classe del plugin implementa la `Funzione` del programma. Con `null` come padre il loader cercherebbe `Funzione` nel `.jar`, la troverebbe (la copia) e la classe del plugin implementerebbe *quella*: il cast a `Funzione` nel Plot fallirebbe con "Funzione cannot be cast to Funzione"; e `@Plot`, che non è nel `.jar`, darebbe `NoClassDefFoundError`.

**Risposta 5.5.** Il file dei servizi ha un nome diverso dal nome binario dell'interfaccia (`jar -list`); il file è nel posto sbagliato (`META-INF/service`, senza `s`, o fuori da `META-INF`); il `.jar` non è fra gli URL del loader (stampare gli URL con `loader.getURLs()`); il file è vuoto o contiene solo commenti. Un nome di classe sbagliato dentro il file, invece, dà `ServiceConfigurationError` durante l'iterazione, non zero provider.

Capitolo 6

# Codice scritto adesso: il compilatore dentro il programma

Con il capitolo 5 il Plot accoglie codice compilato da altri. Resta la prima richiesta del capitolo 1, quella da cui tutto è partito: l'utente scrive `sin(x) * exp(-x / 5)` in un campo e vede la curva. Fra il testo e la curva ci sono tre passaggi, e due li abbiamo già: da un sorgente Java si ottengono dei `.class` (il compilatore), da dei `.class` un loader costruisce una `Class` (capitolo 5), e da una `Class` si ottiene un'istanza di `Funzione` (capitolo 3). Il passaggio nuovo è il primo, e la notizia di questo capitolo è che il compilatore di Java è una classe della libreria standard: si chiama come un metodo, riceve un sorgente, restituisce gli errori come oggetti. Lo useremo per scrivere una piccola libreria, `Compilatore`, che il resto del programma chiama con una riga; poi tradurremo gli errori sulle coordinate dell'utente, diremo con chiarezza che cosa il codice dell'utente può fare nel nostro programma (tutto), vedremo la variante senza file su disco, e chiuderemo il cerchio salvando la funzione compilata come plugin del capitolo 5.

## 6.1 `javac` è una classe

Il package `javax.tools` definisce l'interfaccia `JavaCompiler`, e il JDK ne fornisce l'implementazione, la stessa che il comando `javac` usa. Si ottiene chiamando `getSystemJavaCompiler()` sulla classe `ToolProvider`, e il primo controllo da fare è che non sia `null`: un *JRE*, cioè una distribuzione di Java senza gli strumenti di sviluppo, non ha il compilatore, e un programma che gira su un JRE (o in Eclipse con un JRE invece di un JDK fra le *Installed JREs*) lo scopre così. Il compilatore lavora su `JavaFileObject`, un'astrazione di "file sorgente" che non deve per forza stare su disco: la classe `SimpleJavaFileObject` ha già tutto tranne il contenuto, e una sottoclasse di dieci righe lo prende da una stringa. Gli errori vanno a un `DiagnosticListener`, e `DiagnosticCollector` ne è uno che li raccoglie in una lista. La chiamata vera è `getTask(...)`, che prepara una compilazione, e `call()`, che la esegue e dice se è andata bene. Tutto questo sta nella classe che useremo per il resto del capitolo:

```java
public class Compilatore {

    private final JavaCompiler javac;
    private final Path cartella;
    private final String classPath;
    private int contatore;
    private Path ultimaUscita;

    public Compilatore() throws IOException {
        javac = ToolProvider.getSystemJavaCompiler();
        if (javac == null)
            throw new IllegalStateException("nessun compilatore: serve un JDK");
        cartella = Files.createTempDirectory("compilatore-");
        classPath = System.getProperty("java.class.path");
    }

    /** Compila il sorgente della classe con quel nome binario e la carica. */
    public Class<?> compila(String nomeClasse, String sorgente)
            throws ErroreDiCompilazione {
        JavaFileObject file = new SorgenteInMemoria(nomeClasse, sorgente);
        var diagnostiche = new DiagnosticCollector<JavaFileObject>();
        Path uscita = cartella.resolve("c" + (++contatore));
        ultimaUscita = uscita;
        List<String> opzioni =
                List.of("-d", uscita.toString(), "-classpath", classPath);
        try {
            Files.createDirectories(uscita);
            boolean ok = javac.getTask(null, null, diagnostiche, opzioni, null,
                                       List.of(file)).call();
            if (!ok)
                throw new ErroreDiCompilazione(sorgente,
                                               diagnostiche.getDiagnostics());
            URL[] dove = { uscita.toUri().toURL() };
            ClassLoader padre = getClass().getClassLoader();
            URLClassLoader loader = new URLClassLoader(dove, padre);
            return Class.forName(nomeClasse, true, loader);
        } catch (IOException | ClassNotFoundException e) {
            throw new IllegalStateException("non caricabile: " + nomeClasse, e);
        }
    }
```

Il costruttore trova il compilatore, crea una cartella di lavoro temporanea e legge il class path del programma in esecuzione (la proprietà `java.class.path`: in Eclipse è la cartella `bin` del progetto, in un `.jar` eseguibile è il `.jar`). `compila` prepara il sorgente come `JavaFileObject`, un collettore di diagnostiche, una sottocartella nuova per i `.class` di questa compilazione, e le opzioni, le stesse che si darebbero a `javac` sulla riga di comando: `-d` per la destinazione e `-classpath` perché la classe compilata possa nominare le classi del programma (`Funzione`, `@Plot`). I sei argomenti di `getTask` sono, nell'ordine, dove scrivere i messaggi (`null`: standard error), il gestore dei file (`null`: quello standard, su disco), il destinatario delle diagnostiche, le opzioni, i nomi delle classi per i processori di annotazioni (`null`), e i sorgenti. `call()` restituisce `false` se c'è almeno un errore. La classe continua con il metodo che il Plot chiamerà

e con la classe del sorgente:

```
    /** Come compila, ma restituisce gia' un'istanza del tipo chiesto. */
    public <T> T istanzia(String nomeClasse, String sorgente, Class<T> tipo)
            throws ErroreDiCompilazione {
        Class<?> classe = compila(nomeClasse, sorgente);
        if (!tipo.isAssignableFrom(classe))
            throw new IllegalArgumentException(nomeClasse + ": tipo sbagliato");
        try {
            return tipo.cast(classe.getDeclaredConstructor().newInstance());
        } catch (ReflectiveOperationException e) {
            throw new IllegalStateException("non istanziabile: " + nomeClasse, e);
        }
    }

    /** Un sorgente che sta in una stringa invece che in un file. */
    private static final class SorgenteInMemoria extends SimpleJavaFileObject {
        private final String codice;

        SorgenteInMemoria(String nomeClasse, String codice) {
            super(URI.create("string:///" + nomeClasse.replace('.', '/') +
    ".java"),
                  Kind.SOURCE);
            this.codice = codice;
        }

        @Override
        public CharSequence getCharContent(boolean ignoraErroriDiCodifica) {
            return codice;
        }
    }
}
```

Se `call()` restituisce `false` lanciamo `ErroreDiCompilazione`, un'eccezione controllata nostra che porta con sé il sorgente e la lista delle diagnostiche:

```
public class ErroreDiCompilazione extends Exception {

    private final String sorgente;
    private final List<Diagnostic<? extends JavaFileObject>> diagnostiche;

    public ErroreDiCompilazione(String sorgente,
            List<Diagnostic<? extends JavaFileObject>> diagnostiche) {
        super(diagnostiche.isEmpty() ? "compilazione fallita"
                : "riga " + diagnostiche.get(0).getLineNumber() + ": "
                  + diagnostiche.get(0).getMessage(Locale.ITALIAN));
        this.sorgente = sorgente;
        this.diagnostiche = diagnostiche;
    }

    public String getSorgente() { return sorgente; }
    public List<Diagnostic<? extends JavaFileObject>> getDiagnostiche() {
        return diagnostiche;
    }
}
```

Se invece la compilazione riesce, i `.class` stanno nella sottocartella, e il capitolo 5 fa il resto: un `URLClassLoader` su quella cartella, con il loader del programma come padre, e `forName` con quel loader. `istanzia` aggiunge i passi del capitolo 2, il controllo del tipo e il costruttore, e restituisce direttamente un oggetto del tipo chiesto: è il metodo che il Plot chiamerà. Un primo uso, con un sorgente scritto in un *text block*:

```
Compilatore c = new Compilatore();
String src = """
    package prova;
    public class Saluto {
        public String toString() { return "ciao dal codice compilato adesso"; }
        public static int doppio(int n) { return 2 * n; }
    }
    """;
Class<?> k = c.compila("prova.Saluto", src);
Object o = k.getDeclaredConstructor().newInstance();
System.out.println(o + " / " + k.getMethod("doppio", int.class).invoke(null, 21));
Class<?> k2 = c.compila("prova.Saluto", src.replace("ciao", "salve"));
Object o2 = k2.getDeclaredConstructor().newInstance();
System.out.println(o2 + " / stessa classe? " + (k == k2));
```

```
ciao dal codice compilato adesso / 42
salve dal codice compilato adesso / stessa classe? false
```

La seconda compilazione ha lo stesso nome di classe della prima, e funziona: ogni compilazione ha la sua cartella e il suo loader, quindi `prova.Saluto` esiste due volte, in due loader, come nel capitolo 5. È la ragione per cui il `Compilatore` non usa la cartella `bin` del progetto né il loader dell'applicazione, che caricherebbe il nome una volta sola e poi ignorerebbe ogni versione successiva.

**Osservazione 6.1**

Il costo. La prima compilazione in un processo costa qualche decimo di secondo, perché il compilatore si inizializza e carica le proprie classi; le successive una decina di millisecondi per un sorgente piccolo. Niente che si noti in un programma a riga di comando, ma un listener di Swing che compila blocca l'interfaccia per quel tempo: nel Plot la compilazione andrà in un thread a parte, con il risultato riportato sull'EDT, come insegna *Thread e concorrenza*.

## 6.2 Leggere gli errori

Una `Diagnostic` è un errore o un avviso con tutto ciò che `javac` stamperebbe: `getKind()` (`ERROR`, `WARNING`, ...), `getLineNumber()` e `getColumnNumber()`, `getMessage(Locale)` con il testo, `getSource()` con il file. Un sorgente con due errori:

```
try {
    c.compila("prova.Rotta", """
        package prova;
        public class Rotta {
            int x = "ciao";
            void f() { return 1 }
        }
        """);
} catch (ErroreDiCompilazione e) {
    for (Diagnostic<?> d : e.getDiagnostiche())
        System.out.printf("%s riga %d colonna %d: %s%n", d.getKind(),
                d.getLineNumber(), d.getColumnNumber(), d.getMessage(null));
}
```

```
ERROR riga 4 colonna 24: ';' expected
```

Un solo errore su due, e non il primo: come `javac`, il compilatore si ferma dopo gli errori di sintassi, e gli errori di tipo (la stringa assegnata a un `int`) compaiono solo quando la sintassi è a posto. Chi mostra gli errori all'utente deve aspettarsi di doverlo fare a ondate.

Le righe e le colonne sono quelle del sorgente compilato. Quando il sorgente è scritto dall'utente per intero va bene così; quando è *generato* da un modello attorno a un pezzo scritto dall'utente, come farà il Plot, "riga 9 colonna 20" non dice niente a chi ha scritto una sola riga. La traduzione è aritmetica, a patto di aver progettato il modello perché lo sia: l'espressione dell'utente va su una riga tutta sua, a una colonna nota, e la traduzione sottrae. La sezione seguente mostra il modello; qui il principio.

**Attenzione 6.1: Il sorgente generato va letto**

Quando un sorgente generato non compila e il messaggio non si capisce, il primo gesto è stampare il sorgente. Una versione storica della calcolatrice con gli operatori a runtime generava `return %s:  }` con i due punti al posto del punto e virgola; nessuna espressione dell'utente compilava mai, e l'errore sembrava sempre suo. Per questo `ErroreDiCompilazione` porta con sé `getSorgente()`: nella finestra non si mostra, nel log sì.

## 6.3 Il modello

L'utente scrive un'espressione in `x`; il programma la incastona in una classe che implementa `Funzione`. Il modello è un text block con i segnaposto di `formatted`:

```java
public class Espressione {

    private static final String MODELLO = """
            package utente;
            import it.unibs.pajc.plot.Funzione;
            import it.unibs.pajc.plot.Plot;
            import static java.lang.Math.*;
            @Plot(nome = "%s", xMin = %s, xMax = %s)
            public class %s implements Funzione {
                public double f(double x) {
                    return
            %s
                    ;
                }
            }
            """;
    private static final int RIGA_ESPRESSIONE = 9;       // la riga
    dell'espressione
    private static final String RIENTRO = "            "; // i 12 spazi davanti
    private static int contatore;

    private final String nomeClasse, sorgente;

    public Espressione(String espressione, String nome, double xMin, double xMax)
    {
        nomeClasse = "F" + (++contatore);
        sorgente = MODELLO.formatted(nome.replace("\"", "'"), xMin, xMax,
                                     nomeClasse, RIENTRO + espressione.strip());
    }

    public String nomeBinario() { return "utente." + nomeClasse; }
    public String sorgente() { return sorgente; }

    public Funzione compila(Compilatore compilatore) throws ErroreDiCompilazione {
        return compilatore.istanzia(nomeBinario(), sorgente, Funzione.class);
    }

    /** Il messaggio di una diagnostica, sulle coordinate dell'espressione. */
    public static String traduci(Diagnostic<?> d) {
        long riga = d.getLineNumber(), colonna = d.getColumnNumber();
        String messaggio = d.getMessage(Locale.ITALIAN);
        if (riga == RIGA_ESPRESSIONE)
            long col = Math.max(1, colonna - RIENTRO.length());
            return "colonna " + col + ": " + messaggio;
        if (riga == RIGA_ESPRESSIONE + 1)
            return "alla fine dell'espressione: " + messaggio;
        return "nel modello (riga " + riga + "): " + messaggio;  // errore nostro
    }
}
```

Quattro scelte, ciascuna con un motivo. `import static java.lang.Math.*` permette di scrivere `sin(x)` invece di `Math.sin(x)`: è la sintassi di una lambda, come promesso nel capitolo

1. Il `return` e il punto e virgola stanno su righe proprie, così l'espressione è sola sulla riga 9 con dodici spazi davanti, e un errore *dentro* l'espressione ha riga 9 e una colonna da cui si sottrae 12; un errore di chiusura (una parentesi che manca) finisce sulla riga 10 e si traduce in "alla fine dell'espressione". Il nome della classe è nuovo a ogni `Espressione` (`F1`, `F2`, ...): non servirebbe, con un loader per compilazione, ma rende ogni classe riconoscibile nei messaggi e nei `.jar`. L'annotazione `@Plot` è scritta nel modello con il nome e l'intervallo che l'utente ha dato: la funzione compilata si presenta come tutte le altre, e quando diventerà un plugin il catalogo la leggerà senza sapere da dove viene. Alla prova, con un'espressione giusta e tre sbagliate:

```
sin(x) * exp(-x / 5)     ok: f(1) = 0,6889  [142 ms]
sin(x) * exp(-x / 5      errori:
    colonna 20: ')' or ',' expected
sin(x) + y               errori:
    colonna 10: cannot find symbol
  symbol:   variable y
  location: class utente.F3
System.exit(0)           errori:
    colonna 12: incompatible types: void cannot be converted to double
```

L'ultima riga merita il paragrafo più importante del capitolo. `System.exit(0)` è stato rifiutato, ma non perché sia pericoloso: perché è `void`, e il modello vuole un `double`. Un'espressione come `x + new Object() { { System.exit(0); } }.hashCode()` è un `double` legittimo, compila, e chiude il programma alla prima valutazione. Il codice dell'utente, una volta compilato e caricato, *è* codice del programma: gira nello stesso processo, con gli stessi permessi, può leggere e cancellare file, aprire connessioni, entrare in un ciclo infinito dentro `paintComponent`.

**Osservazione 6.2**

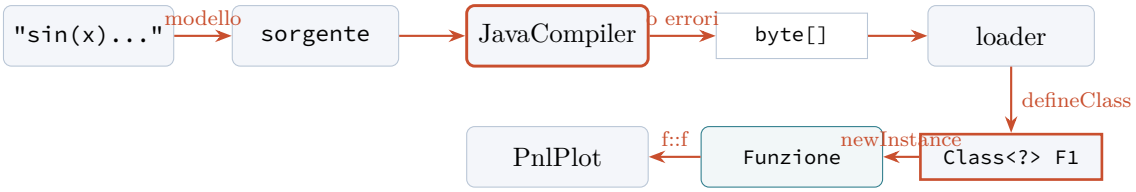
Fino a Java 17 esisteva il `SecurityManager`, un meccanismo con cui si poteva vietare a certo codice certe operazioni; è stato deprecato per la rimozione (JEP 411) perché nessuno riusciva a usarlo correttamente, e in Java 21 è disabilitato per default. Non esiste quindi, nel linguaggio, una *sandbox* per il codice dell'utente. Le difese sono altre: limitare ciò che si *accetta* (un'espressione, non istruzioni: il modello non lascia spazio a dichiarazioni, e un controllo sul testo può rifiutare parole come `System`, `Runtime`, `java.io`), far girare il codice non fidato in un *processo* separato con pochi permessi del sistema operativo, o semplicemente dire all'utente che sta eseguendo codice sul proprio computer, come fa un foglio di calcolo con le macro. Il Plot è nella terza situazione: l'utente compila le proprie espressioni, e un `.jar` in `plugin/` è codice che qualcuno ha scelto di mettere lì, come un programma qualunque.

## 6.4 Tutto in memoria

Il `Compilatore` scrive i `.class` su disco e li rilegge con un `URLClassLoader`. Si può evitare il disco: `getTask` accetta un gestore dei file, e un gestore nostro può consegnare al compilatore, come destinazione dei `.class`, degli oggetti che scrivono in memoria. Dall'altra parte serve un loader che sappia definire una classe da un array di byte: è il metodo `defineClass` di `ClassLoader`, protetto, che ogni loader usa in fondo e che una sottoclasse nostra può chiamare in `findClass`, il metodo che la delega invoca quando nessun padre ha trovato la classe.

```
public Class<?> compila(String nomeClasse, String sorgente)
        throws ErroreDiCompilazione {
    Map<String, ByteArrayOutputStream> classi = new HashMap<>();   // nome -> byte
    DiagnosticCollector<JavaFileObject> diagnostiche = new
    DiagnosticCollector<>();
    StandardJavaFileManager standard =
            javac.getStandardFileManager(diagnostiche, null, null);
    JavaFileManager inMemoria = new ForwardingJavaFileManager<>(standard) {
        @Override
        public JavaFileObject getJavaFileForOutput(Location l, String nome, Kind
    k,
                                                   FileObject s) {
            ByteArrayOutputStream byteClasse = new ByteArrayOutputStream();
            classi.put(nome, byteClasse);
            return new SimpleJavaFileObject(URI.create("bytes:///" + nome), k) {
                @Override
                public OutputStream openOutputStream() { return byteClasse; }
            };
        }
    };
    JavaFileObject file = new SorgenteInMemoria(nomeClasse, sorgente);
    String classPath = System.getProperty("java.class.path");
    List<String> opzioni = List.of("-classpath", classPath);
    boolean ok = javac.getTask(null, inMemoria, diagnostiche, opzioni, null,
                               List.of(file)).call();
    if (!ok)
        throw new ErroreDiCompilazione(sorgente, diagnostiche.getDiagnostics());
    ClassLoader loader = new ClassLoader(getClass().getClassLoader()) {
        @Override
        protected Class<?> findClass(String nome) throws ClassNotFoundException {
            ByteArrayOutputStream byteClasse = classi.get(nome);
            if (byteClasse == null)
                throw new ClassNotFoundException(nome);
            byte[] b = byteClasse.toByteArray();
            return defineClass(nome, b, 0, b.length);
        }
    };
    try {
        return Class.forName(nomeClasse, true, loader);
    } catch (ClassNotFoundException e) {
        throw new IllegalStateException(e);
    }
}
```

`ForwardingJavaFileManager` inoltra tutto al gestore standard (che continua a leggere il class path dal disco) tranne ciò che ridefiniamo: la richiesta di un file di uscita, a cui rispondiamo con un oggetto che scrive in un `ByteArrayOutputStream` tenuto in una mappa per nome binario (una classe con classi annidate produce più voci). Il loader anonimo, figlio del loader del programma, in `findClass` cerca il nome nella mappa e chiama `defineClass`: da quel momento la classe esiste, e `forName` la trova attraverso la delega normale.




Dal testo alla curva: la pipeline del capitolo, con il `.class` che non tocca mai il disco.

Quando conviene? Quando le compilazioni sono molte e piccole (un REPL, le espressioni di un foglio di calcolo), quando il programma non ha un posto dove scrivere, o quando non si vuole lasciare traccia. Per il Plot la versione su disco va bene, e ha un vantaggio: i `.class` sono già lì, pronti per essere messi in un `.jar`.

## 6.5 Dal codice al `.jar`

L'utente ha scritto un'espressione che gli piace e preme *Salva come plugin*. Vogliamo un `.jar` nella cartella `plugin/`, fatto come quello del collega nel capitolo 5: i `.class` con le cartelle dei package, il manifesto, il file dei servizi. `java.util.jar` ha ciò che serve:

```
public class SalvaPlugin {

    public static void scrivi(Path jar, Path cartellaClass, List<String>
    nomiBinari)
            throws IOException {
        Manifest manifesto = new Manifest();
        manifesto.getMainAttributes().put(Attributes.Name.MANIFEST_VERSION,
    "1.0");
        manifesto.getMainAttributes().putValue("Created-By", "Plot");
        try (OutputStream file = Files.newOutputStream(jar);
             JarOutputStream out = new JarOutputStream(file, manifesto);
             Stream<Path> tutti = Files.walk(cartellaClass)) {
            List<Path> file = tutti.filter(p -> p.toString().endsWith(".class"))
                                   .toList();
            for (Path p : file) {
                String voce = cartellaClass.relativize(p).toString();
                out.putNextEntry(new JarEntry(voce.replace('\\', '/')));
                out.write(Files.readAllBytes(p));
                out.closeEntry();
            }
            String servizi = "META-INF/services/" + Funzione.class.getName();
            out.putNextEntry(new JarEntry(servizi));
            out.write(String.join("\n", nomiBinari).concat("\n").getBytes());
            out.closeEntry();
        }
    }
}
```

`JarOutputStream` è uno zip con il manifesto scritto per primo; ogni `putNextEntry` apre una voce, il cui nome è il percorso dentro l'archivio, e deve essere quello relativo alla cartella dei `.class`, con le barre in avanti anche su Windows: `utente/F1.class` per la classe `utente.F1`. L'ultima voce è il file dei servizi, con il nome dell'interfaccia preso da

`Funzione.class.getName()` così da non poterlo sbagliare (l'`attenzione` del capitolo 5). Il `.jar` che ne esce è indistinguibile da uno esportato da Eclipse, e il `Plugin` del capitolo 5 lo carica senza sapere che è nato da un campo di testo:

```
Espressione esp = new Espressione("cos(x) * cos(x)", "coseno quadro", -3.2, 3.2);
esp.compila(c);
Path jar = Path.of("plugin", "cos2.jar");
SalvaPlugin.scrivi(jar, c.ultimaUscita(), List.of(esp.nomeBinario()));
for (Voce v : new Plugin(Path.of("plugin")).ricarica())
    System.out.println("dal jar: " + v + " [" + v.xMin() + ", " + v.xMax() + "]");
```

```
dal jar: coseno quadro [-3.2, 3.2]
```

## 6.6 L'esempio guida: la funzione scritta nel campo

La finestra del Plot guadagna un campo per l'espressione, uno per il nome, due per l'intervallo, e due pulsanti. *Disegna* compila; *Salva come plugin* scrive il `.jar` e ricarica la cartella. La compilazione avviene fuori dall'EDT, in un `SwingWorker`: `doInBackground` compila, `done` torna sull'EDT con il risultato o con l'errore.

```
private final Compilatore compilatore = new Compilatore();
private Espressione ultima;                        // l'ultima compilata con
    successo

private void disegnaEspressione() {
    Espressione esp = new Espressione(txtEspressione.getText(), txtNome.getText(),
                                      leggi(txtDa), leggi(txtA));
    btnDisegna.setEnabled(false);
    lblStato.setText("compilo...");
    new SwingWorker<Funzione, Void>() {
        @Override
        protected Funzione doInBackground() throws ErroreDiCompilazione {
            return esp.compila(compilatore);               // fuori dall'EDT
        }

        @Override
        protected void done() {                            // sull'EDT
            btnDisegna.setEnabled(true);
            try {
                Funzione f = get();
                ultima = esp;
                pnlPlot.setIntervallo(leggi(txtDa), leggi(txtA));
                pnlPlot.setFunzione(f::f);
                lblStato.setText("f(x) = " + txtEspressione.getText());
            } catch (ExecutionException e) {
                if (e.getCause() instanceof ErroreDiCompilazione errore)
                    Diagnostic<?> prima = errore.getDiagnostiche().get(0);
                    lblStato.setText(Espressione.traduci(prima));
                else
                    lblStato.setText("errore: " + e.getCause());
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }.execute();
}

private void salvaComePlugin() {
    if (ultima == null)
        return;
    try {
        String nomeFile = ultima.nomeBinario().replace('.', '-') + ".jar";
        Path jar = Path.of("plugin", nomeFile);
        List<String> nomi = List.of(ultima.nomeBinario());
        SalvaPlugin.scrivi(jar, compilatore.ultimaUscita(), nomi);
        ricarica();                                    // il menu ha una voce in
    piu'
        lblStato.setText("salvata in " + jar);
    } catch (IOException e) {
        lblStato.setText("impossibile salvare: " + e.getMessage());
    }
}
```

`get()` dentro `done` restituisce ciò che `doInBackground` ha restituito, o rilancia la sua eccezione, incartata in una `ExecutionException`; il pattern `instanceof` riconosce nella causa il nostro `ErroreDiCompilazione`, e si mostra la prima diagnostica tradotta sulla colonna dell'espressione. Il resto è ciò che il Plot sapeva già fare: la funzione compilata è una `Funzione` come le altre, e `setFunzione` non sa che cosa le è arrivato. *Salva come plugin* usa la cartella dell'ultima compilazione riuscita, scrive il `.jar` e chiama la `ricarica()` del capitolo 5: la funzione compare nel menù con il nome scritto dall'utente, letto da `@Plot`, e un collega che riceve il file la vede comparire nel suo.

Le tre richieste del capitolo 1 sono soddisfatte, e il programma che le soddisfa è composto di pezzi che abbiamo visto uno alla volta: `forName` e `isAssignableFrom` (capitolo 2), i metodi e i costruttori per reflection (capitolo 3), `@Plot` e un proxy (capitolo 4), i loader e `ServiceLoader` (capitolo 5), il compilatore e il `.jar` (questo capitolo). La palestra, nel capitolo 7, chiede di metterli insieme per intero.

**Ricetta: compilare ed eseguire una stringa**

1. `ToolProvider.getSystemJavaCompiler()`, e un messaggio chiaro se è `null` (JRE);
2. il sorgente in un `SimpleJavaFileObject`; se è generato da un modello, l'utente su una riga sua, a colonna nota;
3. `getTask` con `-d` su una cartella nuova e `-classpath` uguale a `java.class.path`; `DiagnosticCollector` per gli errori;
4. se `call()` è falso, le diagnostiche all'utente tradotte sulle sue coordinate, e il sorgente generato nel log;
5. un `URLClassLoader` nuovo sulla cartella, con il loader del programma come padre; `forName` con quel loader; controllo del tipo, costruttore, cast;
6. fuori dall'EDT se c'è una finestra; e la consapevolezza che il codice compilato è codice del programma.

## Esercizi

**Esercizio 6.1: Il compilatore che non c'è (⋆)**

`ToolProvider.getSystemJavaCompiler()` restituisce `null`. Elencare le cause possibili e, per un programma Swing distribuito agli utenti come `.jar`, dire come si può garantire che il compilatore ci sia.

**Esercizio 6.2: Due compilazioni (⋆)**

Il `Compilatore` riceve due volte di seguito lo stesso nome di classe con sorgenti diversi. Dire quante classi, quanti loader e quante cartelle esistono dopo, e che cosa succederebbe invece se `compila` usasse sempre la stessa cartella e il loader dell'applicazione (`Class.forName(nome)` senza loader).

### Esercizio 6.3: Quale riga (⋆⋆)

Con il modello di `Espressione`, l'utente scrive `sin(x) * (x +`. Dire su quale riga e quale colonna del sorgente generato il compilatore segnala l'errore, che cosa stampa `traduci`, e perché il modello mette il punto e virgola su una riga a parte invece che dopo l'espressione.

### Esercizio 6.4: Il class path sbagliato (⋆⋆)

Si toglie `-classpath` dalle opzioni del `Compilatore`. Dire che cosa succede compilando (a) `prova.Saluto` della sezione 6.1; (b) la classe generata da `Espressione`; e che cosa cambia se il programma gira da Eclipse o da un `.jar` eseguibile.

### Esercizio 6.5: Che cosa può fare (⋆⋆)

Per ciascuna espressione dire se il modello la accetta (compila) e che cosa fa quando il Plot la valuta: (a) `x / 0`; (b) `(double) "ciao".length()`; (c) `System.currentTimeMillis()`; (d) `new java.io.File("dati.txt").delete() ? 1 : 0`; (e) `f(x)`. Quale controllo sul testo fermerebbe (d), e quale espressione lo aggirerebbe?

*I problemi che seguono chiedono un programma completo, in ordine crescente di difficoltà. Le soluzioni sono nel repository delle soluzioni del corso, nel progetto e nel package indicati.*

### Esercizio 6.6: Diagnostiche (⋆)

Scrivere un programma che legge un file `.java` dato sulla riga di comando, lo compila con `javax.tools` in una cartella temporanea e stampa tutte le diagnostiche, errori e avvisi (opzione `-Xlint:all`), nel formato `tipo riga:colonna messaggio`, seguite dalla riga di sorgente incriminata con un `^` sotto la colonna. Se compila, stampa i nomi delle classi prodotte (anche quelle annidate).

Soluzione: progetto `sol-dxtr-01-c06`, package `it.unibs.pajc.redolfi.e06`.

### Attenzione 6.2: Colonne e tabulazioni

`getColumnNumber()` conta i caratteri, e una tabulazione vale uno; il `^` sotto la colonna va allineato sostituendo le tabulazioni della riga con lo stesso numero di spazi, o ricopiandole davanti al `^`.

### Esercizio 6.7: La calcolatrice che impara (⋆⋆)

Una calcolatrice a riga di comando conosce `+`, `-`, `*` e `/` come `DoubleBinaryOperator` in una mappa. Il comando `nuovo <nome> <espressione in a e b>` (per esempio `nuovo hyp sqrt(a*a + b*b)`) compila l'espressione in una classe che implementa `DoubleBinaryOperator` e la aggiunge alla mappa; da quel momento `hyp 3 4` stampa 5. Gli errori di compilazione si mostrano tradotti sull'espressione. Il comando `salva` scrive i sorgenti degli operatori in una cartella, e all'avvio la calcolatrice li ricompila.

Soluzione: progetto `sol-dxtr-01-c06`, package `it.unibs.pajc.redolfi.e07`.

**Esercizio 6.8: Un piccolo REPL (⋆⋆)**

Scrivere un ciclo leggi-valuta-stampa per espressioni Java su `double`: ogni riga è o un'assegnazione `nome = espressione` o un'espressione da stampare, e le variabili assegnate restano visibili alle righe successive. Ogni riga diventa una classe compilata al volo; le variabili vivono come campi `static` di una classe di contesto generata e ricompilata quando se ne aggiunge una, oppure in una `Map<String, Double>` che la classe generata riceve.
*Suggerimento: la seconda strada è più semplice: il modello dichiara `double nome = ctx.get("nome");` per ogni variabile nota prima dell'espressione.*

Soluzione: progetto `sol-dxtr-01-c06`, package `it.unibs.pajc.redolfi.e08`.

**Esercizio 6.9: Tutto in memoria, misurato (⋆⋆⋆)**

Completare il `CompilatoreInMemoria` della sezione 6.4 in una classe con la stessa interfaccia del `Compilatore` (`compila`, `istanzia`, `ErroreDiCompilazione`) e gestire correttamente una classe con classi annidate e lambda (più `.class` per una compilazione). Scrivere poi un programma che compila cento piccole classi con l'uno e con l'altro, misurando il tempo della prima compilazione e la media delle successive, e commentare i numeri.

Soluzione: progetto `sol-dxtr-01-c06`, package `it.unibs.pajc.redolfi.e09`.

## Risposte

**Risposta 6.1.** Il programma gira su un JRE (una distribuzione senza `jdk.compiler`), o su un'immagine ridotta creata con `jlink` che non include quel modulo, o in Eclipse con un JRE configurato come ambiente di esecuzione. Per garantirlo: il `.jar` si distribuisce insieme a un JDK, o con un'immagine `jlink` che include `jdk.compiler`; e il programma controlla il `null` all'avvio e lo dice, invece di fallire al primo *Disegna*.

**Risposta 6.2.** Due classi con lo stesso nome, in due loader diversi, in due cartelle (`c1` e `c2`). Con una sola cartella e il loader dell'applicazione, il secondo `call()` sovrascriverebbe il `.class`, ma `forName` restituirebbe la classe caricata la prima volta: la seconda versione non si vedrebbe mai, senza nessun errore.

**Risposta 6.3.** L'espressione è sulla riga 9; la parentesi aperta non chiusa fa segnalare l'errore dove il compilatore trova il punto e virgola, cioè sulla riga 10, e `traduci` stampa "alla fine dell'espressione: ')' expected". Con il punto e virgola subito dopo l'espressione l'errore starebbe sulla riga 9, ma a una colonna oltre la fine del testo dell'utente, meno leggibile; e un'espressione che contiene un commento `//` inghiottirebbe il punto e virgola.

**Risposta 6.4.** (a) Compila: `prova.Saluto` usa solo la libreria standard. (b) Non compila: `Funzione` e `Plot` non si trovano ("package it.unibs.pajc.plot does not exist"). Senza `-classpath`, il compilatore usa il class path del processo *solo* se la proprietà `java.class.path` è quella standard; in Eclipse lo è (`bin`), da un `.jar` eseguibile è il `.jar`, in entrambi i casi funziona per caso: meglio passarlo sempre.

**Risposta 6.5.** (a) Compila; vale `Infinity` o `NaN`, e il pannello interrompe la linea. (b) Compila; vale 4. (c) Compila; è un `long`, convertito in `double`: una retta quasi orizzontale enorme. (d) Compila; alla prima valutazione cancella il file, se esiste. (e) Non compila: `f` è il metodo che la contiene, con un parametro, e `f(x)` è una ricorsione infinita che compila e manda in `StackOverflowError` al primo ridisegno. Un controllo che rifiuta `java.io` fermerebbe (d), aggirato con `Class.forName("java.io.File")` e la reflection, o con un `import` che non c'è; la lista nera non chiude mai.

Capitolo 7

# Palestra

Nessuna teoria nuova. Sette programmi in ordine crescente di difficoltà, ciascuno con l'indicazione di che cosa allena, che arrivano a chiudere le tre richieste del capitolo 1: la funzione scritta in un campo, salvata e condivisa, e la cartella dei plugin. I primi due sono nello stile delle prove scritte: prevedere l'output di un frammento che usa la reflection, e interrogare una gerarchia. I tre seguenti mettono insieme annotazioni, loader e compilatore in programmi che somigliano a quelli veri: un runner che trova i test da solo, una calcolatrice grafica che impara, un ispettore di finestre. Gli ultimi due sono programmi completi con una struttura da progettare, e il settimo è il Plot. Dopo ogni enunciato, l'errore che su quel tipo di problema si fa più spesso; le soluzioni, con i commenti sulle scelte, sono nel repository delle soluzioni del corso, progetto `sol-d06-c07`. Il consiglio di sempre: prima di scrivere, dire a parole quale oggetto `Class` si sta interrogando, chi l'ha caricato e che cosa può lanciare ogni chiamata; la soluzione solo dopo.

## Problemi

**Esercizio 7.1: Prevedere la reflection (⋆)**

*Allena: `getMethods` e `getDeclaredMethods`, `invoke`, le eccezioni, nello stile dei quesiti a scelta multipla.* Per ciascuno dei quattro frammenti dire, senza eseguire, che cosa stampa o quale eccezione lancia e da dove; poi scrivere un programma che li esegue uno alla volta, ciascuno in un `try`/`catch`, e confrontare.

```
// (a)
Arrays.stream(JComponent.class.getMethods())
      .filter(m -> m.getName().startsWith("setV"))

    .map(Method::getName).distinct().sorted().forEach(System.out::println);
// (b)
System.out.println(String.class.getDeclaredMethods().length
                   == String.class.getMethods().length);
// (c)
Method m = Math.class.getMethod("max", int.class, int.class);
System.out.println(m.invoke(null, 3, 7.0));
// (d)
Class<?> k = Class.forName("java.util.ArrayList");
Object o = k.getDeclaredConstructor().newInstance();
System.out.println(o.getClass().getMethod("size").invoke(o));
```

Soluzione: progetto `sol-dxtr-01-c07`, package `it.unibs.pajc.redolfi.e01`.

**Attenzione 7.1: L'argomento incartato nel tipo sbagliato**

In (c) il secondo argomento è un `Double`, e il parametro è `int`: `invoke` non converte fra tipi incartati diversi, e il frammento lancia `IllegalArgumentException` ("argument type mismatch") prima ancora di chiamare `max`. Chi risponde "7" ha applicato le conversioni del compilatore, che qui non c'è.

**Esercizio 7.2: La scheda del bimondo (⋆)**

*Allena: `Class`, le gerarchie `sealed`, i costruttori per reflection, `invoke` su un metodo d'interfaccia.* Scrivere un'interfaccia `sealed Shape2d` con `area()` e `perimeter()` e tre record che la implementano (`Circle`, `Rectangle`, `Triangle` con i tre lati). Un programma stampa, partendo da `Shape2d.class` e senza nominare le sottoclassi, la gerarchia (per ogni classe permessa: nome, componenti del record, costruttori), poi legge un file di testo con una forma per riga (nome semplice e misure), costruisce ogni forma con il costruttore giusto, e chiama `area` e `perimeter` *per reflection* (con `getMethod` su `Shape2d`), stampando una tabella e i totali. Una riga sbagliata si segnala e si salta.

Soluzione: progetto `sol-dxtr-01-c07`, package `it.unibs.pajc.redolfi.e02`.

**Attenzione 7.2: I record non hanno il costruttore senza argomenti**

`getDeclaredConstructor()` senza tipi cerca il costruttore vuoto, che un record non ha: `NoSuchMethodException`. I tipi dei parametri sono quelli dei componenti, e `getRecordComponents()` li dà nell'ordine giusto, pronti per `getDeclaredConstructor(tipi)`.

**Esercizio 7.3: Il runner che trova le classi (⋆⋆)**

*Allena: annotazioni, `URLClassLoader`, l'ispezione senza inizializzare.* Il mini-JUnit del capitolo 4 riceve il nome di una classe. Scrivere un runner che riceve invece una *cartella* di `.class` (la `bin` di un progetto Eclipse, o un `.jar`), la percorre, ricava il nome binario di ogni classe dal percorso del file, la carica con un `URLClassLoader` *senza inizializzarla*, e se ha almeno un metodo `@Test` la esegue con il runner del capitolo 4 (copiato nel package). Alla fine un riepilogo per classe e totale. Le classi che non si lasciano caricare (dipendenze mancanti) si segnalano e si saltano.

Soluzione: progetto `sol-dxtr-01-c07`, package `it.unibs.pajc.redolfi.e03`.

**Attenzione 7.3: Il nome dal percorso**

`bin/it/unibs/pajc/Prova$Interna.class` è la classe `it.unibs.pajc.Prova$Interna`: si toglie la radice, si sostituisce il separatore con il punto, si toglie `.class`, e il `$` resta. Chi lo sostituisce con un punto ottiene un nome che `forName` non trova; chi costruisce il percorso con `/` su Windows non trova i file.

### Esercizio 7.4: La calcolatrice che impara (⋆⋆)

*Allena: il compilatore a runtime dentro un'interfaccia grafica, gli errori tradotti, la persistenza dei sorgenti.* Una calcolatrice Swing con un display, i tasti delle cifre e quelli di `+`, `-`, `*`, `/`, ciascuno legato a un `DoubleBinaryOperator`. Un pulsante *Nuovo operatore* apre una finestra di dialogo modale con il nome, il simbolo e il corpo dell'operazione in `a` e `b` (per esempio `Math.pow(a, b)`): *OK* compila il corpo in una classe che implementa `DoubleBinaryOperator`, e se compila un tasto nuovo compare nella tastiera, altrimenti il dialogo mostra l'errore sulla riga del corpo e resta aperto. Gli operatori nuovi si salvano come sorgente in una cartella `operatori/` e si ricompilano all'avvio. La compilazione non blocca l'interfaccia.

Soluzione: progetto `sol-dxtr-01-c07`, package `it.unibs.pajc.redolfi.e04`.

### Attenzione 7.4: Il dialogo che si chiude comunque

Il dialogo modale non deve chiudersi su *OK* finché il corpo non compila: chi chiude e poi mostra l'errore in un `JOptionPane` costringe l'utente a riscrivere tutto. E il refuso storico di questa calcolatrice (un `:` al posto di `;` nel modello) si trova in un minuto stampando il sorgente generato, e in un'ora altrimenti.

### Esercizio 7.5: L'ispettore di finestre (⋆⋆⋆)

*Allena: getter e setter per reflection, le conversioni dei tipi, un albero di oggetti.* Dato un `JFrame` qualunque (per esempio il Plot stesso), scrivere una finestra *Ispettore* che mostra a sinistra l'albero dei componenti (`JTree`: ogni `Container` con i suoi figli, etichettati con il nome semplice della classe e il `getName()` se c'è) e a destra, per il componente selezionato, una tabella con le proprietà lette dai getter pubblici senza parametri (`getX`, `isX`): nome, tipo, valore. Le proprietà di tipo `String`, `int`, `boolean` e `Color` che hanno anche un setter si possono modificare nella tabella, e la modifica si applica subito (con `Color` espresso come `#RRGGBB`). Un pulsante *Aggiorna* rilegge l'albero.
*Suggerimento: alcuni getter lanciano o bloccano (per esempio `getGraphics` su un componente non mostrato, `getToolkit`): un elenco di nomi da saltare, e ogni `invoke` in un `try`.*

Soluzione: progetto `sol-dxtr-01-c07`, package `it.unibs.pajc.redolfi.e05`.

### Attenzione 7.5: Leggere le proprietà dal thread sbagliato

L'ispettore legge e scrive proprietà di componenti Swing: va fatto sull'EDT, come ogni accesso a un componente. Un ispettore che percorre l'albero in un thread a parte "funziona quasi sempre". E un setter chiamato per reflection con il valore del tipo sbagliato (`"12"` invece di `12`) lancia `IllegalArgumentException`: la conversione è nostra.

### Esercizio 7.6: Un gestore di plugin riusabile (⋆⋆⋆)

*Allena: loader, `ServiceLoader`, scarico, errori isolati, un'API pulita.* Scrivere una classe generica `Plugin<T>` che riceve l'interfaccia del servizio e una cartella, e offre: `ricarica()` che restituisce la lista dei provider trovati in tutti i `.jar`, caricando *ogni `.jar` con un loader proprio* così che un `.jar` rotto (file dei servizi sbagliato, classe che non si istanzia, `NoClassDefFoundError`) produca una voce in un elenco di errori e non fermi gli altri; `errori()` con quell'elenco; una `WeakReference` per ogni loader precedente e un metodo `scaricati()` che dice quanti dei loader vecchi sono stati raccolti dal garbage collector. Provarla con tre `.jar`, uno rotto in ciascun modo, e con una ricarica dopo aver sostituito un `.jar`.

Soluzione: progetto `sol-dxtr-01-c07`, package `it.unibs.pajc.redolfi.e06`.

### Attenzione 7.6: Un loader per tutti i `.jar`

Con un unico `URLClassLoader` su tutti i `.jar` un errore in uno di essi ferma l'iterazione di `ServiceLoader` a metà, e i `.jar` dopo quello non vengono letti. Un loader per `.jar` isola gli errori, ma le classi di `.jar` diversi non si vedono fra loro: va detto nella documentazione, perché è una scelta.

### Esercizio 7.7: Il Plot completo (⋆⋆⋆)

*Allena: tutto; chiude il problema d'apertura.* Il Plot con tutte le versioni dei capitoli 2–6 messe insieme in un programma solo, pulito:

- il menù con le funzioni di `Math`, le classi interne e i metodi annotati `@Plot`, ciascuna con il suo intervallo, e la riga di stato con le valutazioni contate da un proxy;
- il campo dell'espressione con nome e intervallo, *Disegna* che compila fuori dall'EDT e mostra l'errore sulla colonna giusta, con il campo colorato;
- *Salva come plugin* che scrive il `.jar` in `plugin/` e lo fa comparire nel menù; la cartella letta all'avvio e con *Ricarica*;
- un messaggio chiaro all'avvio se il compilatore non c'è, con il Plot che funziona lo stesso per tutto il resto;
- un menù *Aiuto* che dice all'utente che le espressioni e i plugin sono codice eseguito con i suoi permessi.

Soluzione: progetto `sol-dxtr-01-c07`, package `it.unibs.pajc.redolfi.e07`.

### Attenzione 7.7: Le versioni vecchie che restano vive

Dopo ogni *Ricarica* il menù deve contenere *solo* oggetti della ricarica nuova: una lista a cui si aggiunge senza svuotare tiene in vita tutti i loader precedenti, e dopo venti ricariche il programma ha venti copie di ogni classe. E la `Funzione` che il pannello sta disegnando è un oggetto della ricarica precedente finché l'utente non ne sceglie un'altra: è giusto così, ma va saputo.

Capitolo 8

# Dove andare adesso

La cassetta degli attrezzi è completa per i programmi del corso: `Class` e i membri, `invoke`, le annotazioni e i proxy, i class loader, `ServiceLoader`, il compilatore e il `.jar`. Questo capitolo indica le direzioni in cui il discorso continua, con un assaggio di ciascuna e qualcosa da provare subito. I testi per ogni sentiero sono in fondo.

## 8.1 `java.lang.invoke`: chiamate scoperte a runtime, veloci come le dirette

Il capitolo 3 ha misurato `Method.invoke`: alcuni nanosecondi, contro una frazione per la chiamata diretta, perché ogni `invoke` ricontrolla argomenti e accesso. I *method handle* JAVA 7 spostano i controlli al momento della ricerca: un `MethodHandle` è un riferimento a un metodo già verificato, con un tipo esatto, che il JIT tratta quasi come una chiamata normale.

```
MethodHandles.Lookup lookup = MethodHandles.lookup();
MethodType tipo = MethodType.methodType(double.class, double.class);
MethodHandle sin = lookup.findStatic(Math.class, "sin", tipo);
double y = (double) sin.invokeExact(1.0);        // nessun incartamento, nessun
    Object[]
```

`invokeExact` vuole esattamente i tipi dichiarati, e per questo è veloce. Sui method handle sono costruite le lambda: quando il compilatore vede `x -> x * 2` non genera una classe, ma un'istruzione `invokedynamic` che, al primo uso, chiede a `LambdaMetafactory` di fabbricare la classe della lambda a partire da un method handle al corpo. È il motivo per cui `getClass()` di una lambda mostra un nome come `PlotApp$$Lambda/0x...`, generato al momento. Da provare: riscrivere `incarta` del catalogo con un `MethodHandle` e rifare la misura della sezione 3.5.

## 8.2 Classi nascoste e bytecode generato

`Lookup.defineHiddenClass` JAVA 15 definisce una classe da un array di byte, come il nostro `defineClass` del capitolo 6, ma *nascosta*: non ha un nome che `forName` possa trovare, non è visibile agli altri, e viene scaricata appena non serve. È ciò che la JVM usa per le lambda e i proxy. Per generare i byte, invece di passare dal compilatore, le librerie scrivono bytecode direttamente: ASM è la più bassa, ByteBuddy la più comoda, e con esse un proxy può estendere una *classe* e non solo implementare interfacce (Mockito, Hibernate, Spring lo fanno così). Dal Java 24 il JDK ha la propria *Class-File API* (`java.lang.classfile`) per leggere e scrivere `.class` senza librerie esterne. Da provare: aprire un `.class` con `javap -c -p` e leggere il bytecode di `f` della `Gaussiana`.

## 8.3 Agenti e `Instrumentation`

Un *agente Java* è un `.jar` con una classe che la JVM chiama prima del `main` (opzione `-javaagent:agente.jar`) passandole un oggetto `Instrumentation`: con esso si può registrare un *trasformatore* che riceve i byte di ogni classe mentre viene caricata e può modificarli. È così che i profiler misurano i tempi dei metodi senza toccare il codice, che i framework di copertura dei test sanno quali righe sono state eseguite, e che un debugger ricarica una classe modificata. Un agente di venti righe che stampa il nome di ogni classe caricata è il modo più diretto di vedere la delega dei loader del capitolo 5 al lavoro.

## 8.4 Il sistema dei moduli

Il capitolo 3 ha incontrato l'incapsulamento forte: `setAccessible` fallisce sui privati del JDK. La regola generale è dei *moduli* JAVA 9: un modulo dichiara in `module-info.java` quali package *esporta* (visibili al compilatore) e quali *apre* (visibili alla reflection profonda). Il codice senza `module-info`, come il nostro, vive nel modulo senza nome, aperto a tutto; una libreria modulare può aprire un package solo a un framework (`opens it.unibs.dati to com.fasterxml.jackson.databind`). A runtime `Class.getModule()`, `Module.isOpen(pkg, altro)` e `Module.addOpens` permettono di interrogare e, con i permessi giusti, allargare queste aperture. Da provare: `String.class.getModule()` e `PlotApp.class.getModule()`, e vedere perché il secondo si chiama "unnamed module".

## 8.5 Dove la reflection lavora per noi, e dove non lavora più

JUnit 5 cerca i `@Test` come il nostro runner, con un motore che scopre le classi nel class path; Spring costruisce il grafo degli oggetti leggendo `@Component` e `@Autowired`, e avvolge i servizi in proxy per le transazioni; Jackson e Gson convertono oggetti in JSON e viceversa con `getDeclaredFields` e `setAccessible`; Hibernate fa lo stesso con le tabelle; Eclipse stesso è un sistema di plugin, con un loader per plugin (OSGi). Nello stesso tempo la tendenza recente va nella direzione opposta: la reflection costa all'avvio e non si presta alla compilazione anticipata (GraalVM *native image* deve conoscere in anticipo ogni classe che verrà ispezionata), e i framework più nuovi (Micronaut, Quarkus, Dagger) leggono le annotazioni a *tempo di compilazione*, con un *processore di annotazioni*, e generano il codice che i precedenti costruivano a runtime. Un processore di annotazioni è una classe che il compilatore chiama (lo stesso `JavaCompiler` del capitolo 6, sesto argomento di `getTask`) e che vede le dichiarazioni annotate come un albero: da provare, un processore che per ogni `@Plot` genera il file dei servizi, così da non scriverlo mai a mano.

## 8.6 Java chiama il C

L'ultima direzione è fuori dalla JVM. La *Foreign Function & Memory API* JAVA 22 permette di chiamare una funzione C compilata in una libreria condivisa, cercandola per nome a runtime (`SymbolLookup`) e costruendo un method handle con la firma dichiarata (`Linker.downcallHandle`): la stessa idea di `getMethod` e `invoke`, applicata a codice che non è Java e che non ha un oggetto `Class`. Con Java 25 è nel suo stato definitivo, e un `strlen` chiamato da Java con dieci righe è il modo più concreto di capire che cosa la reflection ha in comune con il collegamento dinamico dei sistemi operativi.

## Nelle altre dispense

*Le interfacce in Java moderno* (le interfacce come contratto, le gerarchie `sealed` che il capitolo 2 ha letto per reflection); *Dal nome alla funzione* (le lambda e le method reference, e il perché un `Method` si incarta in una lambda); *Interfacce grafiche con Swing* (il pannello del Plot, i dialoghi della calcolatrice che impara, l'albero dell'ispettore); *Thread e concorrenza* (il `SwingWorker` con cui il Plot compila fuori dall'EDT).

## Riferimenti

- Javadoc dei package `java.lang.reflect`, `java.lang.annotation`, `java.lang.invoke`, `javax.tools` e `java.util.jar`, e delle classi `Class`, `ClassLoader` e `ServiceLoader`.
- *The Java Language Specification*, cap. 9.6 (annotazioni) e cap. 12 (esecuzione: caricamento, collegamento, inizializzazione).
- Oracle, *The Java Tutorials*: "The Reflection API" e "Annotations".
- J. Bloch, *Effective Java*, 3a ed.: item 39 (preferire le annotazioni alle convenzioni di nome), 65 (preferire le interfacce alla reflection), 59 (conoscere la libreria).
- JEP 396 e 403 (incapsulamento forte degli interni del JDK), JEP 411 (deprecazione del `SecurityManager`), JEP 371 (classi nascoste), JEP 454 (Foreign Function & Memory API).
- JUnit 5, *User Guide*, e il codice sorgente del suo motore: un runner vero, leggibile.